

Introduction to Algorithm

---

# 기초 알고리즘

네트워크 플로우 개념

# 네트워크 플로우 문제



## 단순한 연결 vs 용량의 제한

- '어떻게 빨리 갈까?' 에 대한 연결문제 → DFS, BFS, 다익스트라
- 하지만, 현실세계에서의 연결문제는 더 복잡하고 다양
  - 아무리 빠른 길이라도 차선이 하나 뿐이라면?
  - 이동거리가 길더라도, 차선이 더 많은 고속도로라면?

→ '경로의 길이' 가 아니라, '통로의 너비(용량)' 가 중요



## 네트워크 플로우가 필요한 순간

- 물류센터에서 상품을 보낼 때 → 도로의 차선 수, 컨테이너 용량
- 통신망에서 트래픽을 분산할 때 → 통신선의 대역폭
- 중앙 상수도에서 지역으로 보내는 파이프를 설계 할 때 → 파이프의 지름

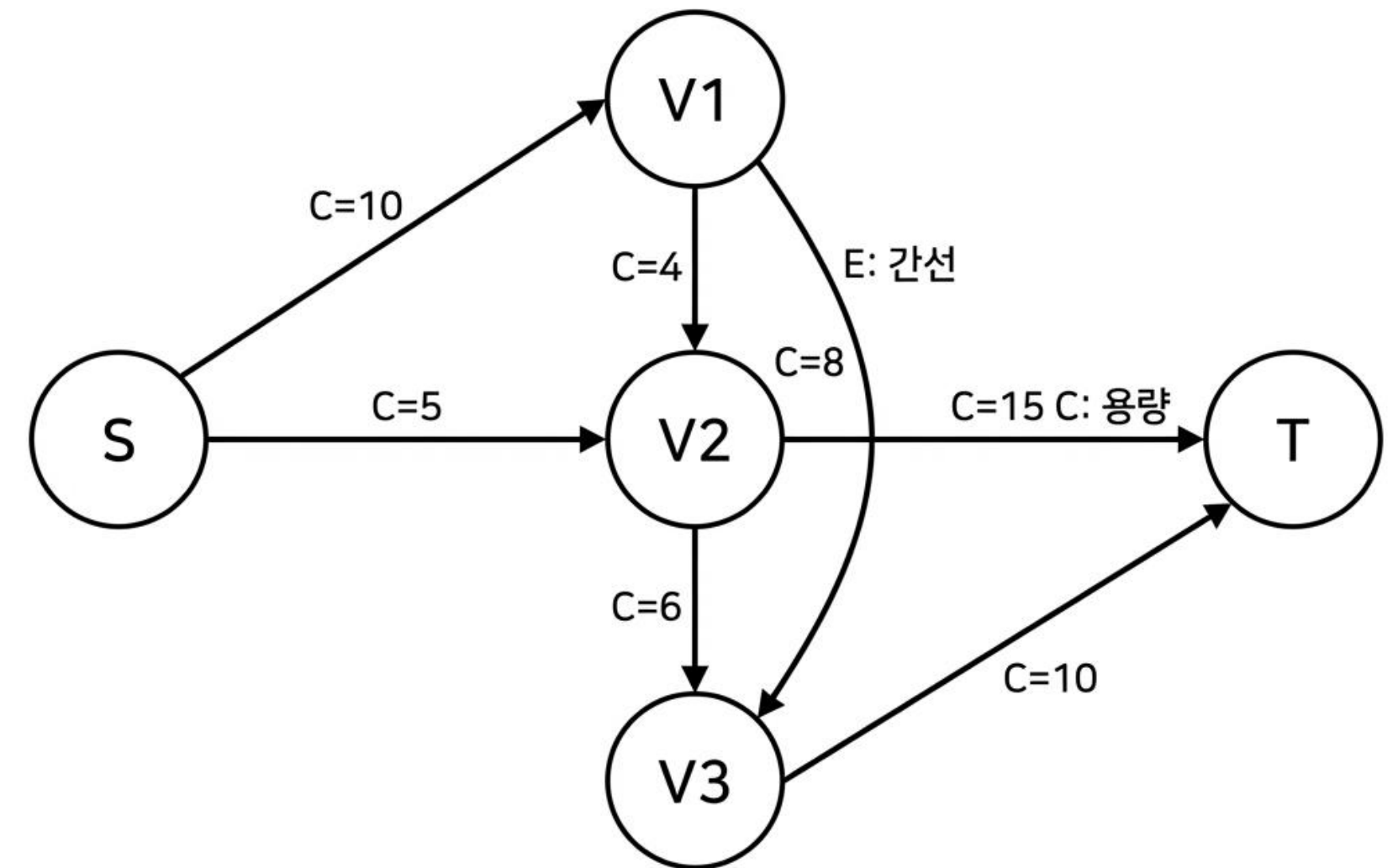
→ '제한된 자원(용량)' 안에서 '흐름' 을 최대화 해야 한다

# 유량 그래프 (Flow Network)



## 유량 그래프의 핵심 요소

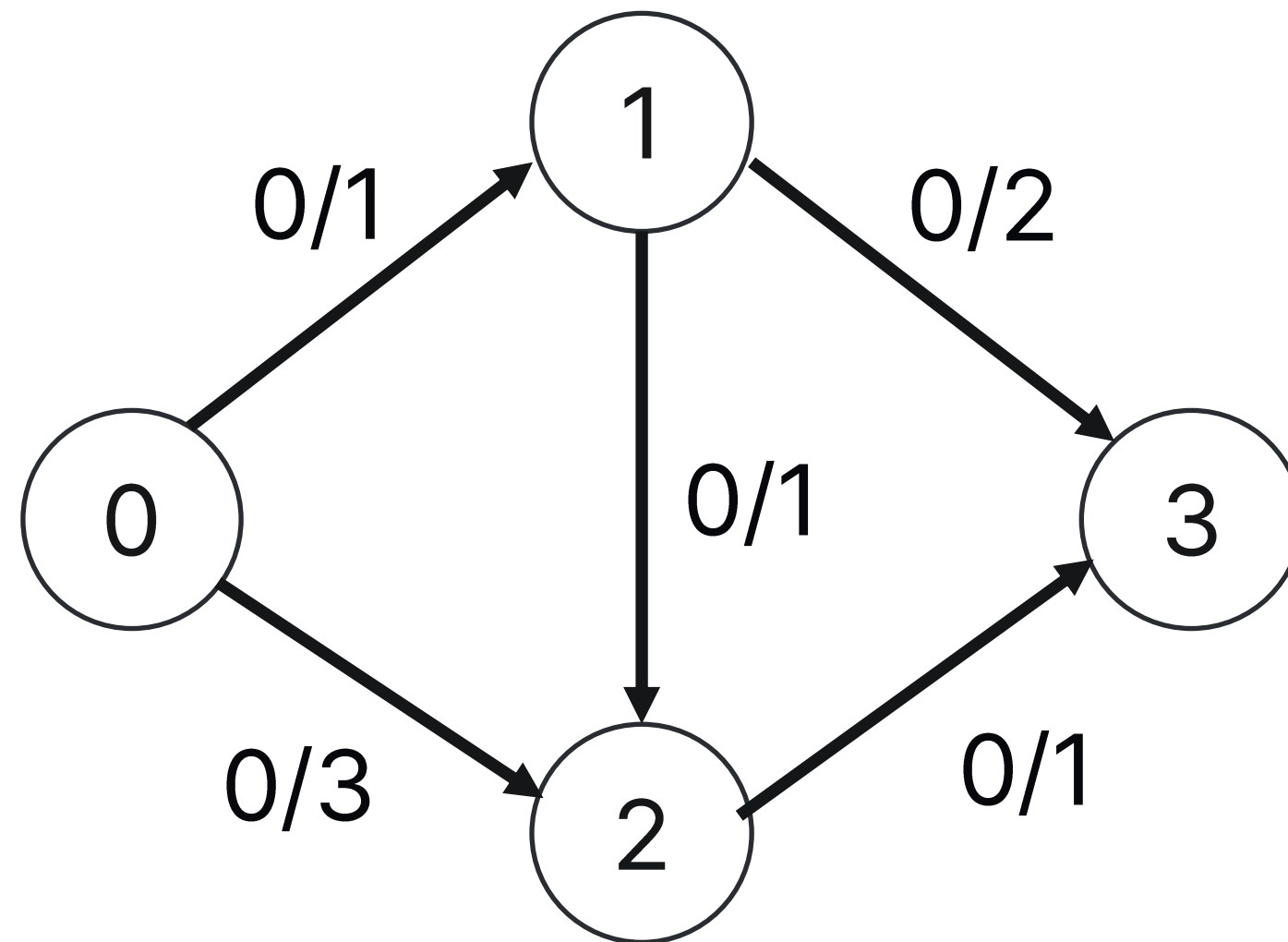
- 정점(V): 데이터나 물이 지나가는 교차점
- 간선(E): 흐름이 전달되는 통로
- 용량(C): 간선이 버틸 수 있는 최대 유량
- 소스(S): 유량이 시작되는 공급원
- 싱크(T): 유량이 최종적으로 모이는 수요처





## 유량 그래프 표기법

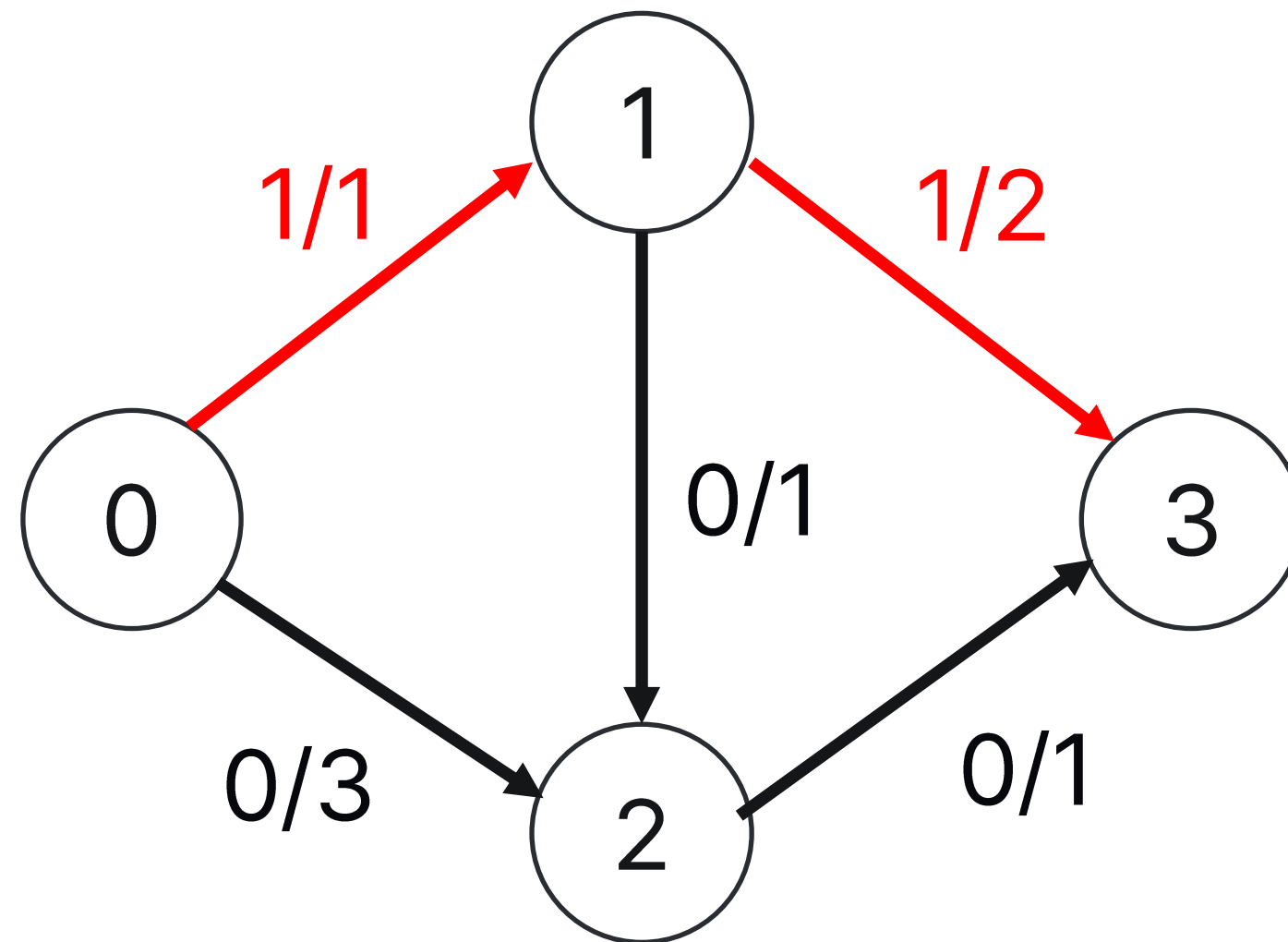
- $A / B$ : 최대 'B' 만큼 보낼 수 있는 간선에 현재 'A'가 흐르고 있음





## 유량 그래프 표기법

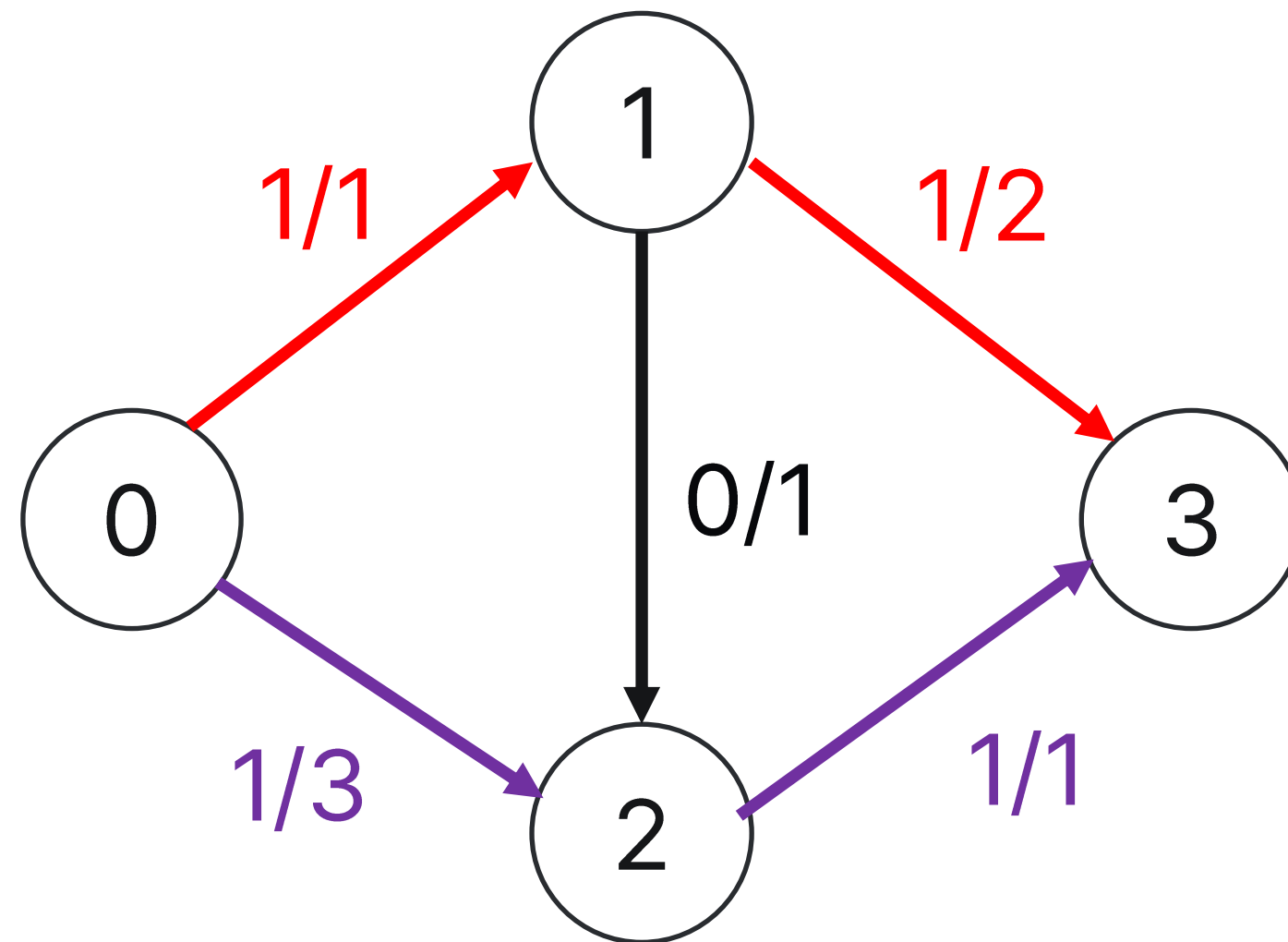
- $A / B$ : 최대 'B' 만큼 보낼 수 있는 간선에 현재 'A'가 흐르고 있음





## 유량 그래프 표기법

- $A / B$ : 최대 'B' 만큼 보낼 수 있는 간선에 현재 'A'가 흐르고 있음



# 최대 유량 (Max Flow)



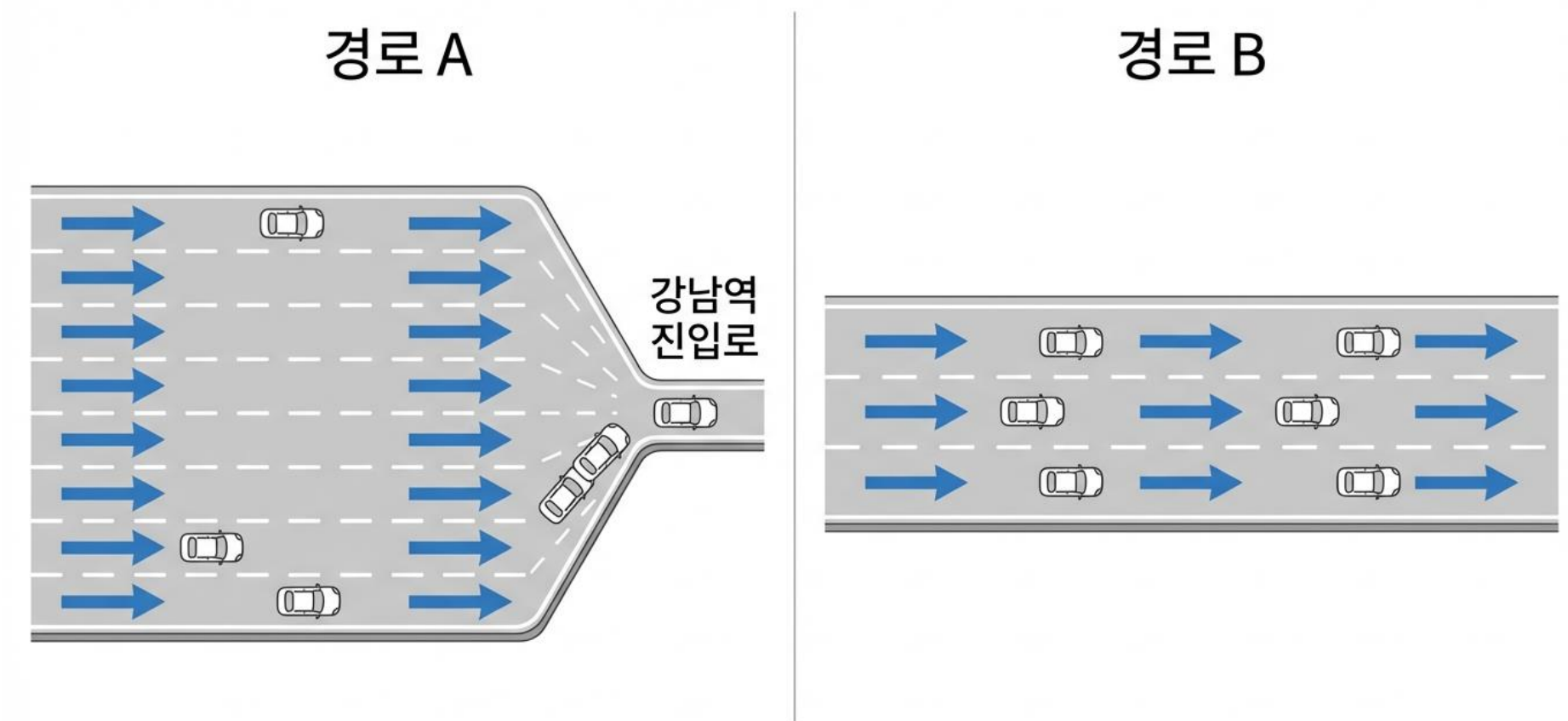
## 최대유량(Max Flow) 문제란

- 소스(Source)에서 생성된 모든 유량이 싱크(Sink)로 들어갈 때, 그 합을 최대화하는 상태 찾기
- 즉, 소스에서 쿼쿼 쏟아부어서 싱크에 최대한 많은 양이 도착하게 만드는 게 목표
- 단, 중간 노드에서 유량이 사라지거나 새로 생성되지 않는다는 전제가 필요



## 최대유량(Max Flow) 문제 예시

- 출발지(Source)에서 강남역(Sink)까지 최대한 많은 차를 보내야 한다. 아래 A, B 경로 중 어느 경로를 선택해야 할까?
- 그래프 구성
  - 경로 A: 8차선 고속도로지만, 마지막에 강남역 진입로가 1차선인 경우
  - 경로 B: 전체가 3차선으로 일정한 국도





## 최대유량(Max Flow) 문제 예시

- 출발지(Source)에서 강남역(Sink)까지 최대한 많은 차를 보내야 한다. 아래 A, B 경로 중 어느 경로를 선택해야 할까?
  - 그래프 구성
    - 경로 A: 8차선 고속도로지만, 마지막에 강남역 진입로가 1차선인 경우
    - 경로 B: 전체가 3차선으로 일정한 국도
- ➔ 아무리 고속도로가 8차선이어도 마지막 진입로가 1차선이면, 그 길로는 한 번에 1대의 차밖에 지나가지 못함. 즉, 전체 흐름은 가장 좁은 구간(병목)에 의해 결정.



## 유량의 제약조건( Flow Constraints)

### [조건 1] 용량 제한 속성

모든 간선에서 흐르는 유량은 해당 간선의 용량을 초과할 수 없습니다.

" 파이프 용량이 넘쳐 흐르면, 파이프는 터집니다"



## 유량의 제약조건( Flow Constraints)

### [조건 2] 유량 보존 속성

소스와 싱크를 제외한 모든 정점으로 들어온 유량의 합은 나가는 유량의 합과 같아야 합니다.

“중간 파이프는 유량을 저장하거나 소모하지 않습니다”

# 잔여 그래프 (Residual Graph)



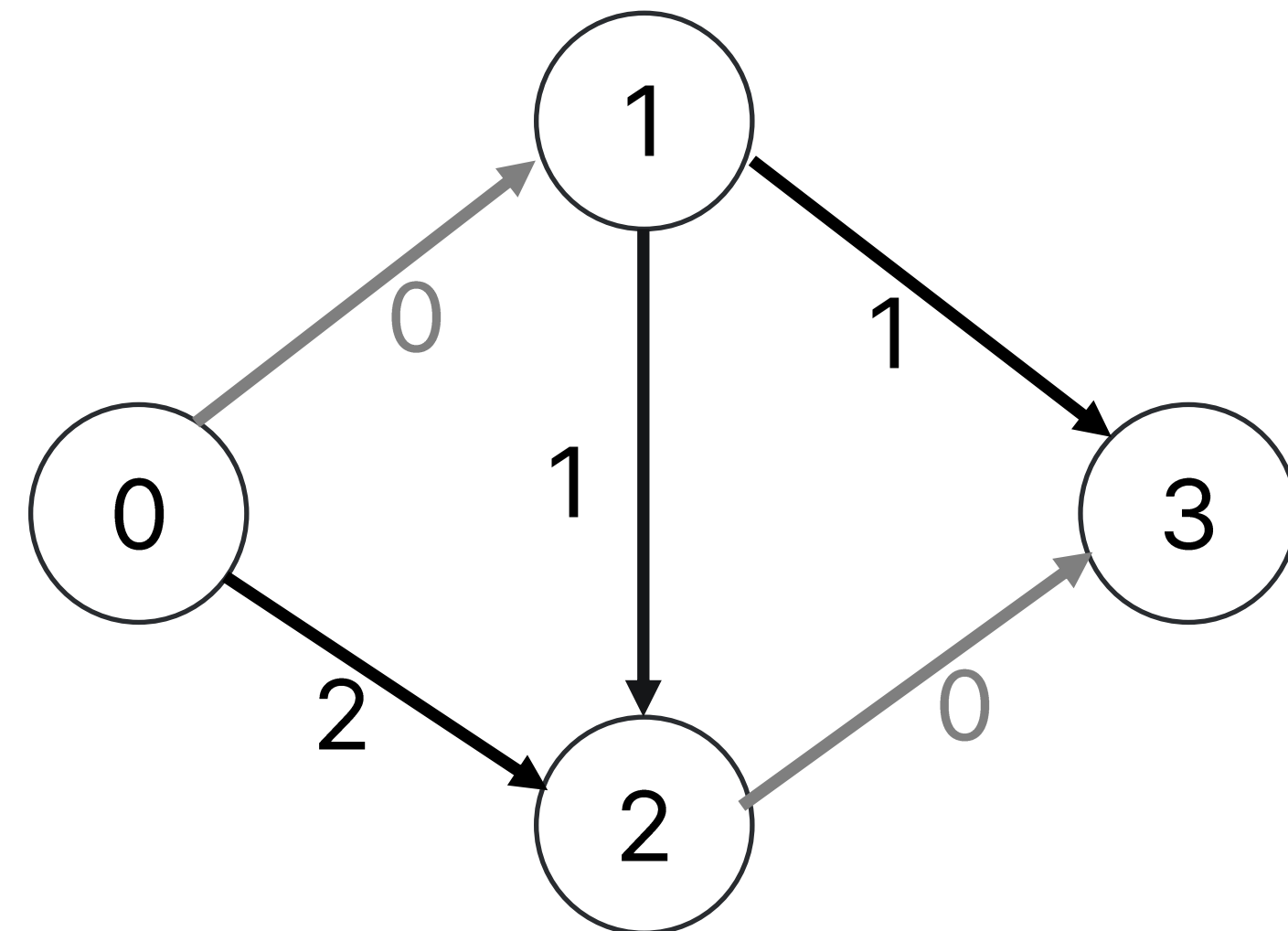
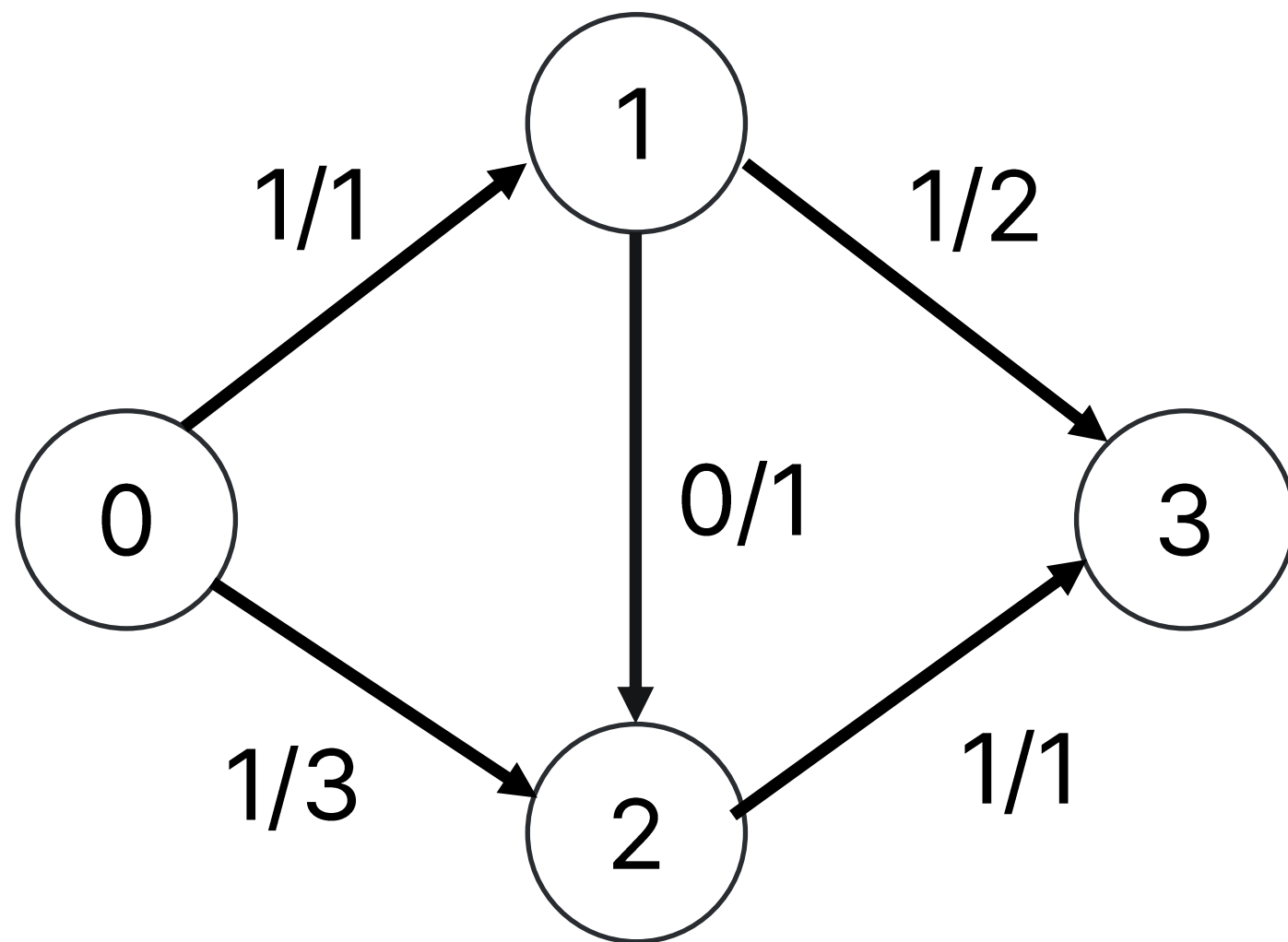
### 잔여 그래프 (Residual Graph) 란?

- 유량을 흘리고 난 뒤, 추가로 흘릴 수 있는 여유 공간("앞으로 더 보낼 수 있는 양")을 나타내는 가상의 그래프
- 현재 상태에서 '수정 가능성'을 탐색하기 위한 지도 역할
- 역방향 간선: 기존 유량을 '거꾸로 보내는' 가상의 간선
  - $A \rightarrow B$ 로 5를 보냈다면,  $B \rightarrow A$ 로 5만큼 "취소할 수 있는 용량" 추가
  - 역방향 간선은 '이전에 내린 결정을 되돌릴 수 있는 기회'를 제공



## 잔여 그래프 표기법

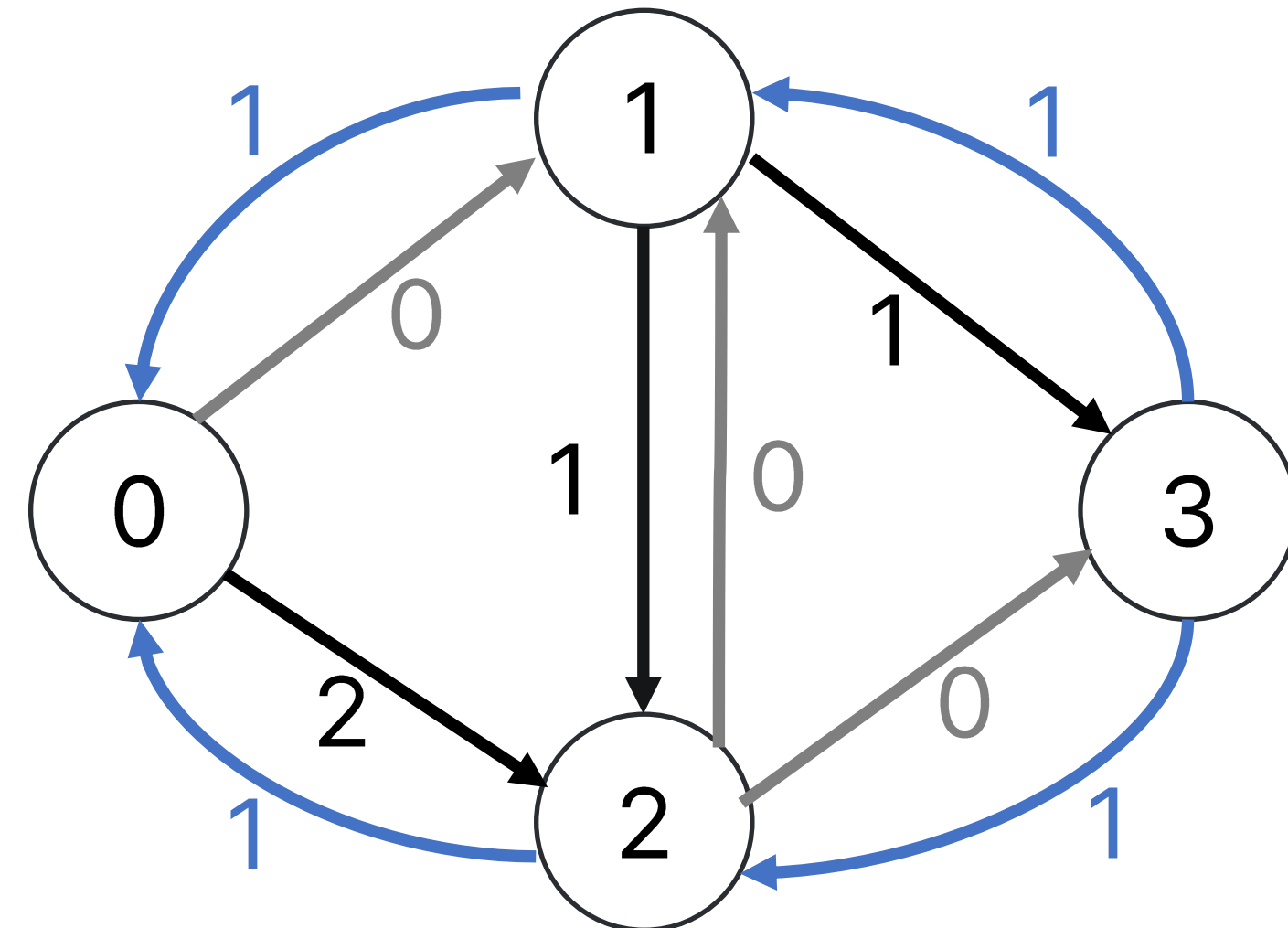
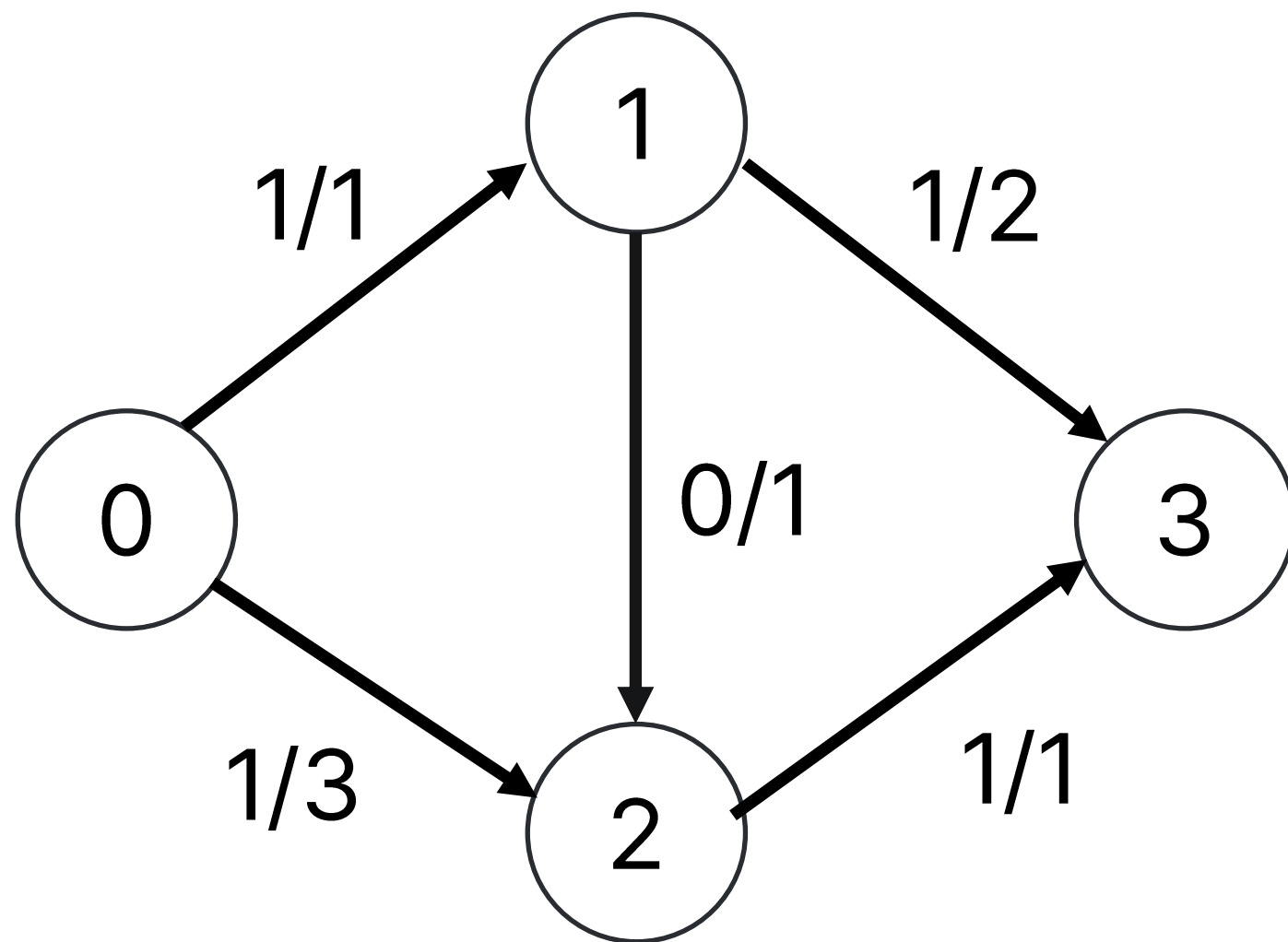
- 정방향: 유량을 흘리고 난 뒤, 추가로 흘릴 수 있는 여유 공간 (수정 가능성)





## 잔여 그래프 표기법

- 역방향: 기존 유량을 '거꾸로 보내는' 가상의 간선



# 증가 경로 (Augmenting Path)



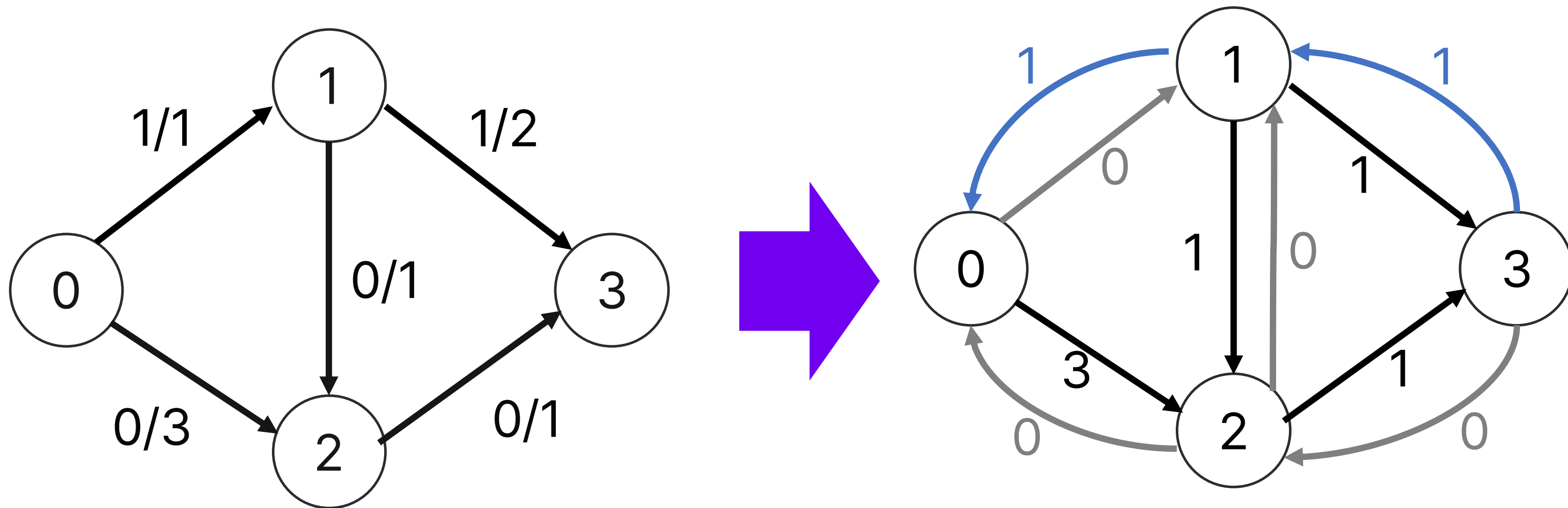
## 증가 경로(Augmenting Path) 란?

- 잔여 그래프에서, 소스에서 싱크까지 **잔여 용량이 남아있는 간선들**로만 이루어진 경로
- '증가 경로' 가 존재한다면, 전체 유량을 더 늘릴 수 있다는 증거
- 선택된 증가경로에 포함된 간선들 중, '가장 작은 잔여용량' 이 해당 경로를 통해 추가 할 수 있는 최대 유량 → **경로의 최대치는 최소치가 결정**



## 증가 경로(Augmenting Path) 란?

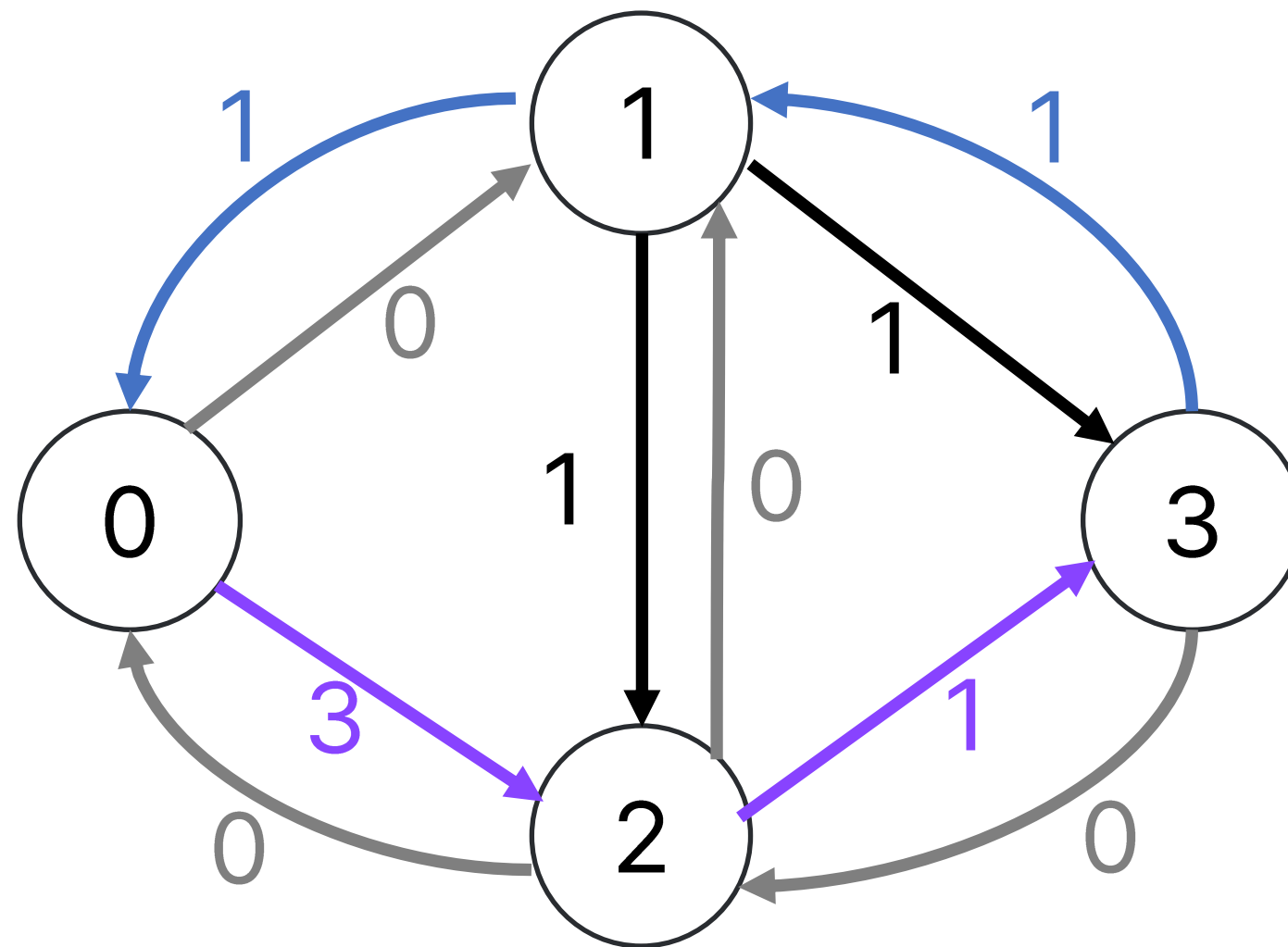
- 유량 그래프(왼쪽) 를 잔여 그래프(오른쪽) 로 변경





## 증가 경로(Augmenting Path) 란?

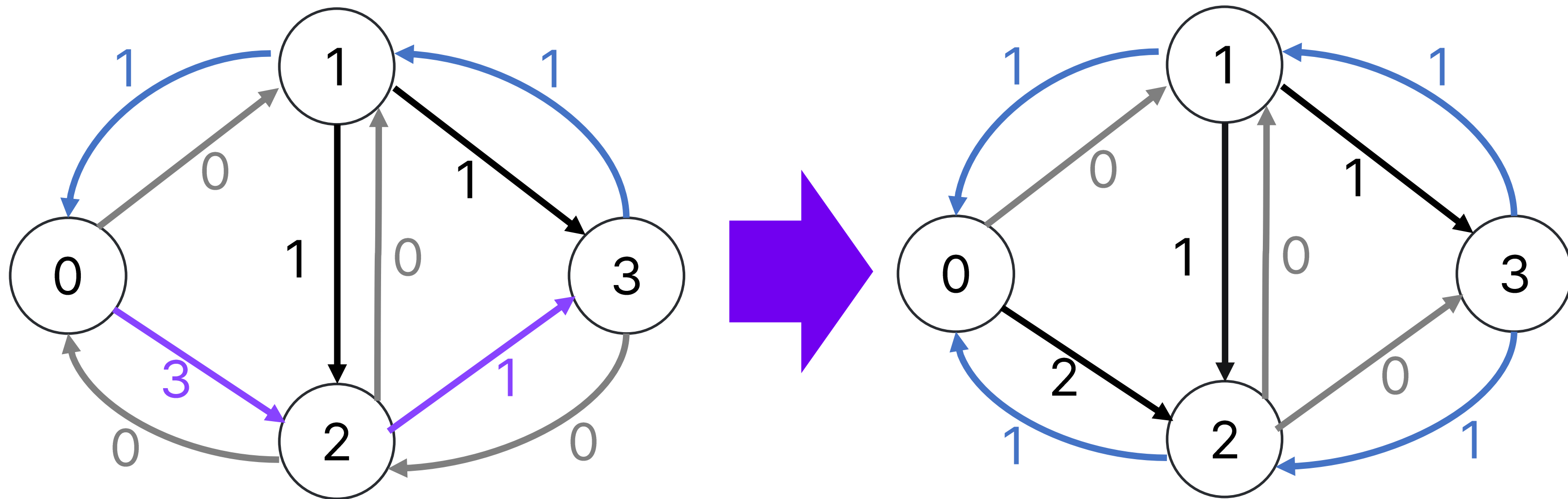
- 아래 예시에서, ' $0 \rightarrow 2 \rightarrow 3$ ' 으로 이루어진 증가경로 존재
- 병목 용량(최소 잔여 용량)은 ' $1$ '





## 증가 경로(Augmenting Path) 란?

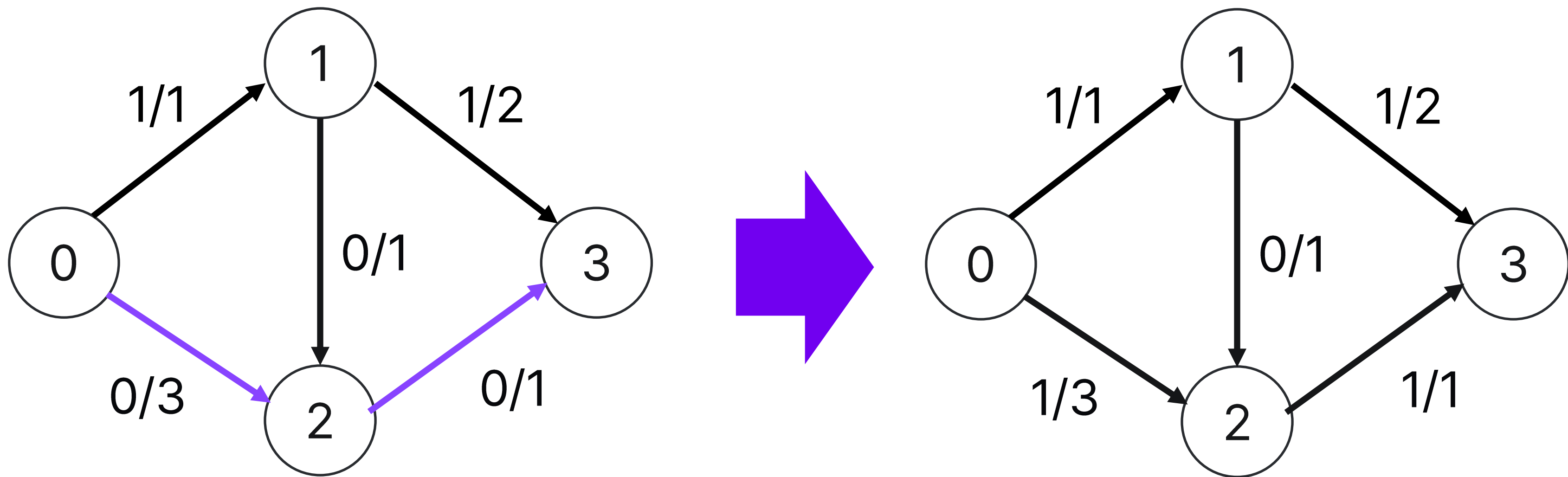
- 발견된 증가 경로를 통해 유량(최소 잔여용량: 1)을 흘리고 나면 잔여그래프는 다음과 같이 변경 됨





## 증가 경로(Augmenting Path) 란?

- 발견된 증가 경로를 통해 유량(최소 잔여용량: 1)을 흘리고 나면 유량그래프는 다음과 같이 변경 됨



# Ford-Fulkerson 알고리즘



## Ford-Fulkerson 알고리즘이란?

- 네트워크 플로우 그래프에서 소스부터 싱크까지 더 이상 증가 경로가 존재하지 않을 때까지 유량을 반복적으로 추가하여, **최대 유량을 구하는 기법**
- 현재 상태에서 추가로 보낼 수 있는 양을 나타내는 잔여 그래프를 활용하며, 이미 흐르고 있는 유량을 되돌릴 수 있는 **역방향 간선 개념을 통해 최적의 해를 찾아냄**



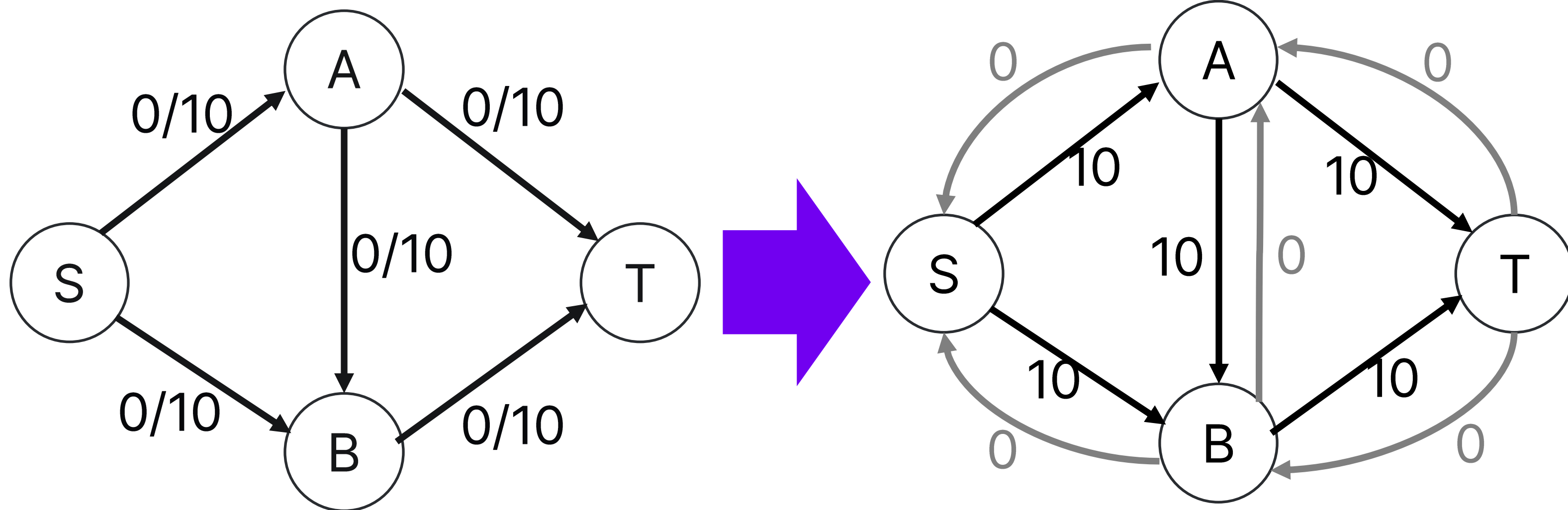
## Ford-Fulkerson 알고리즘 절차 → 반복적인 개선!

1. 처음엔 모든 유량을 0으로 설정합니다.
2. 잔여 그래프에서 증가 경로를 하나 찾습니다.
3. 찾은 경로의 병목 용량만큼 유량을 흘립니다.
4. 경로가 더 이상 없을 때까지 반복합니다.



## Ford-Fulkerson 알고리즘 예시

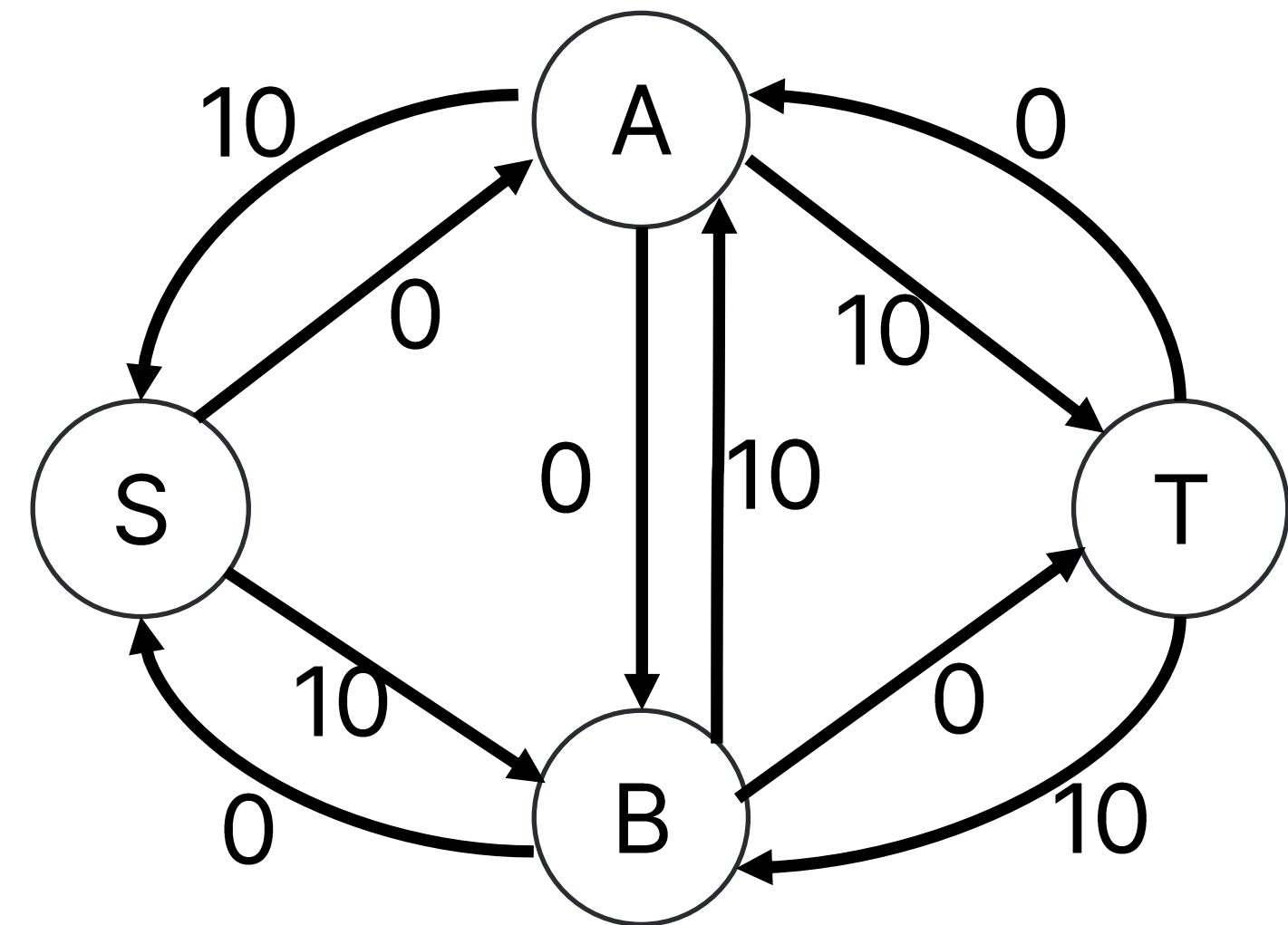
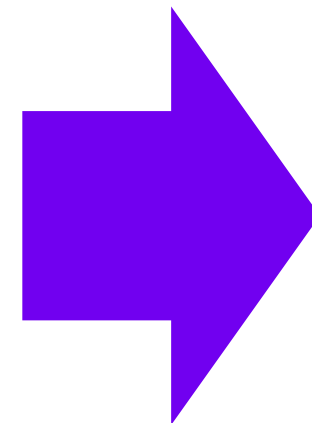
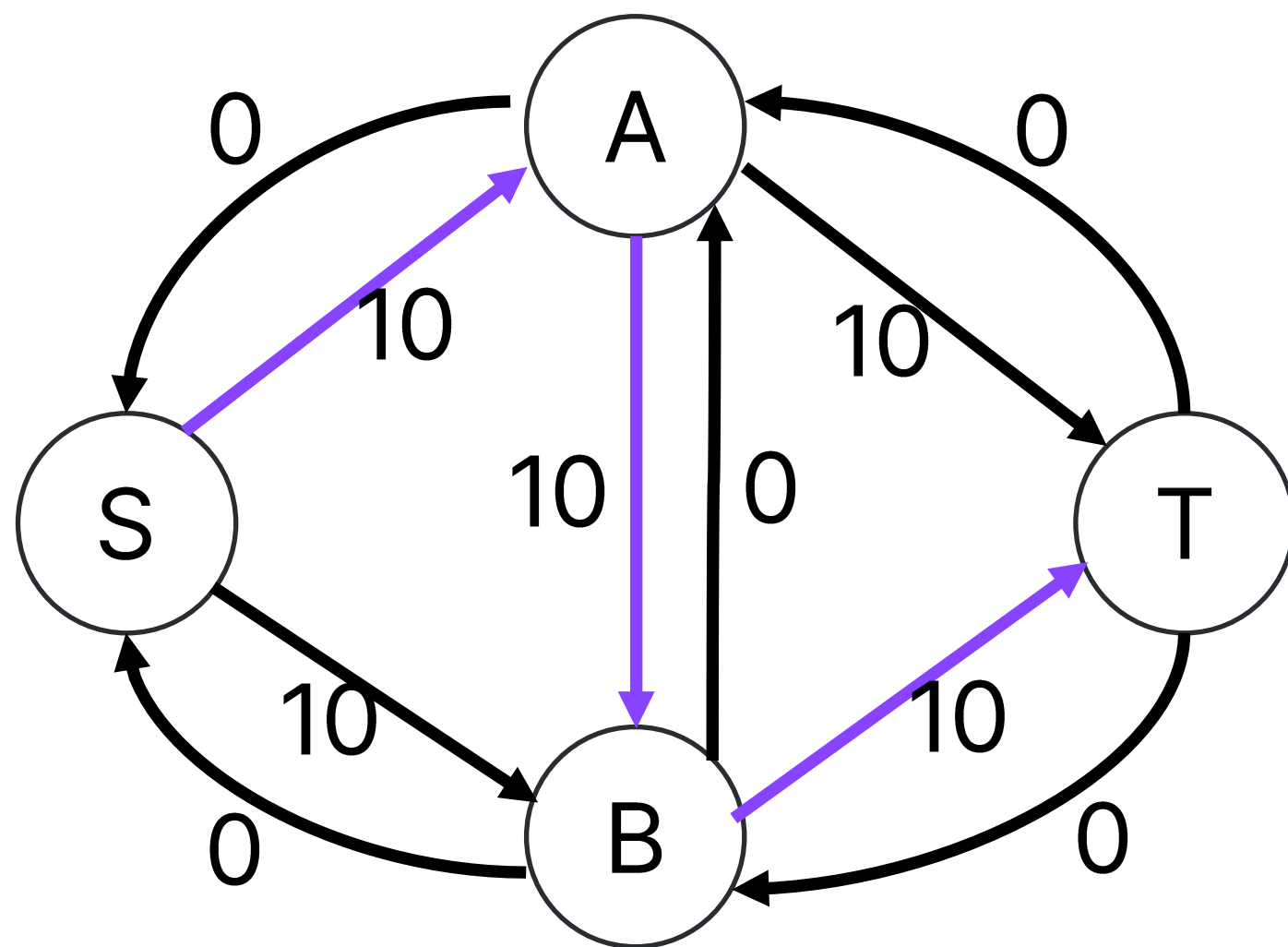
초기 상태: 잔여 그래프로 변환





## Ford-Fulkerson 알고리즘 예시

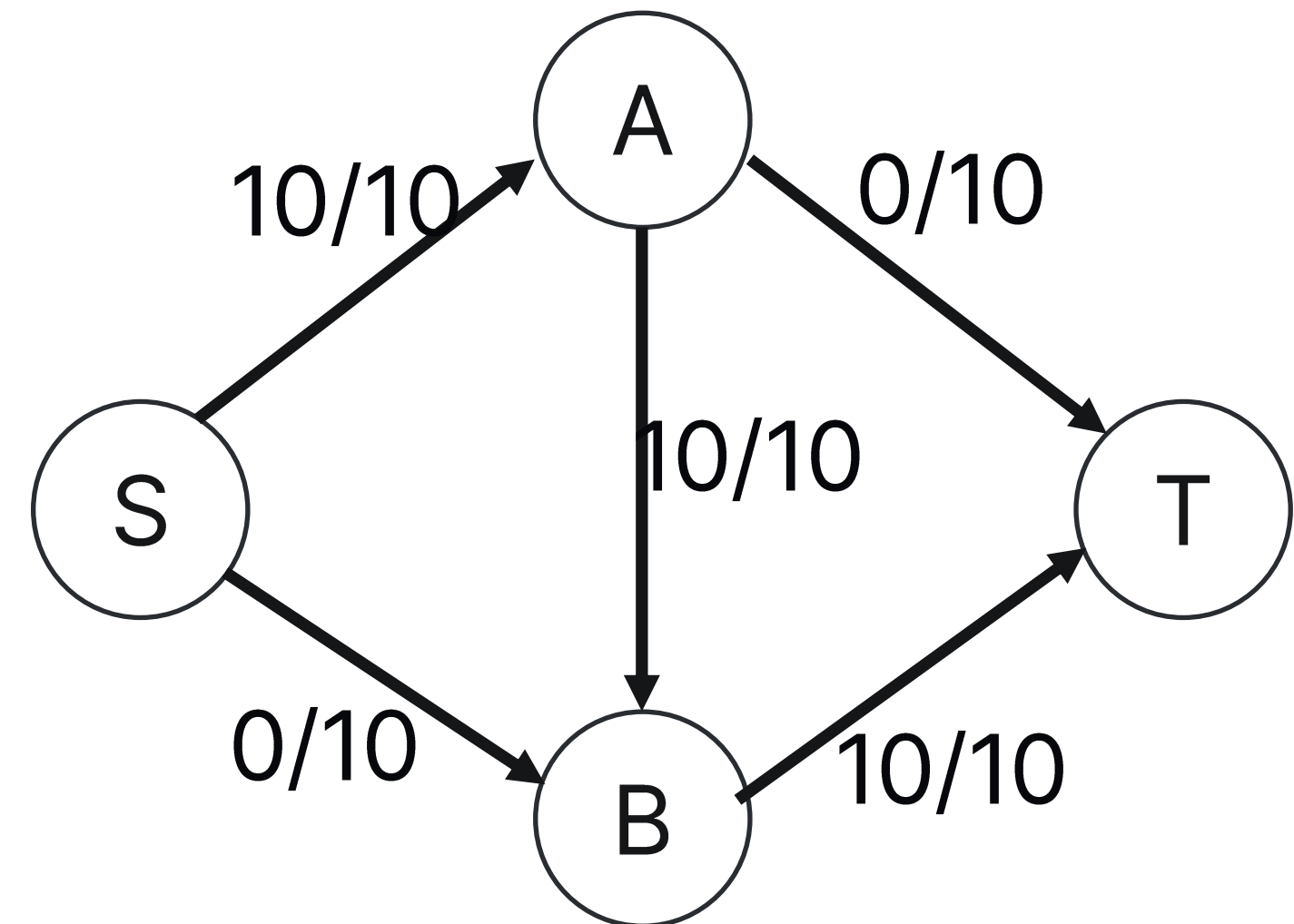
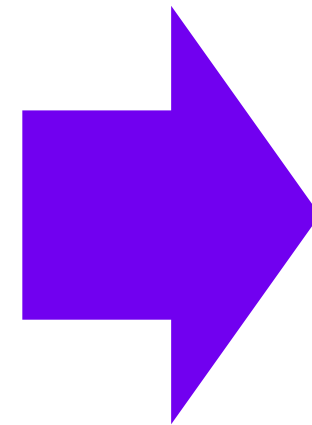
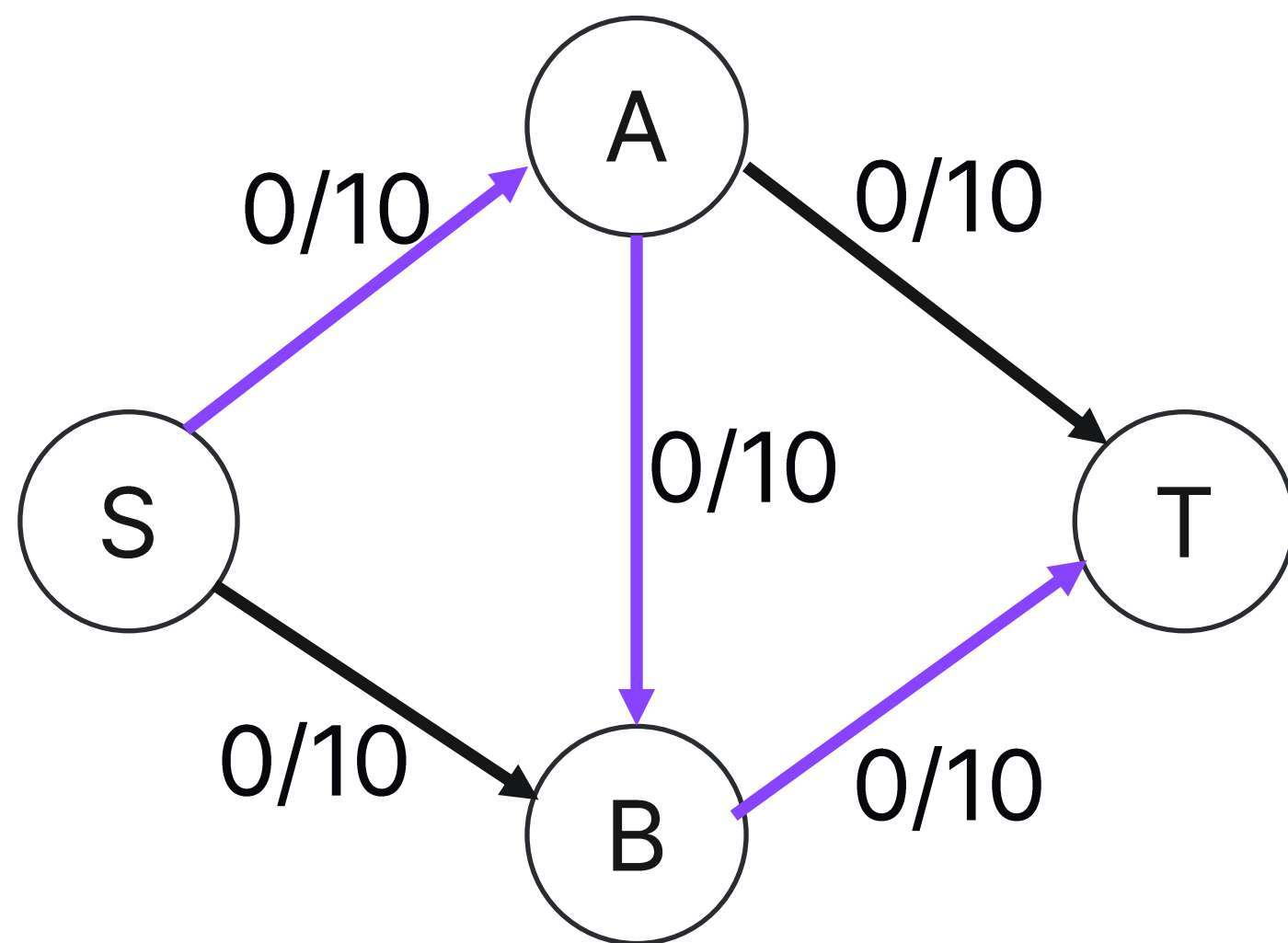
증가경로 발견:  $S \rightarrow A \rightarrow B \rightarrow T$





## Ford-Fulkerson 알고리즘 예시

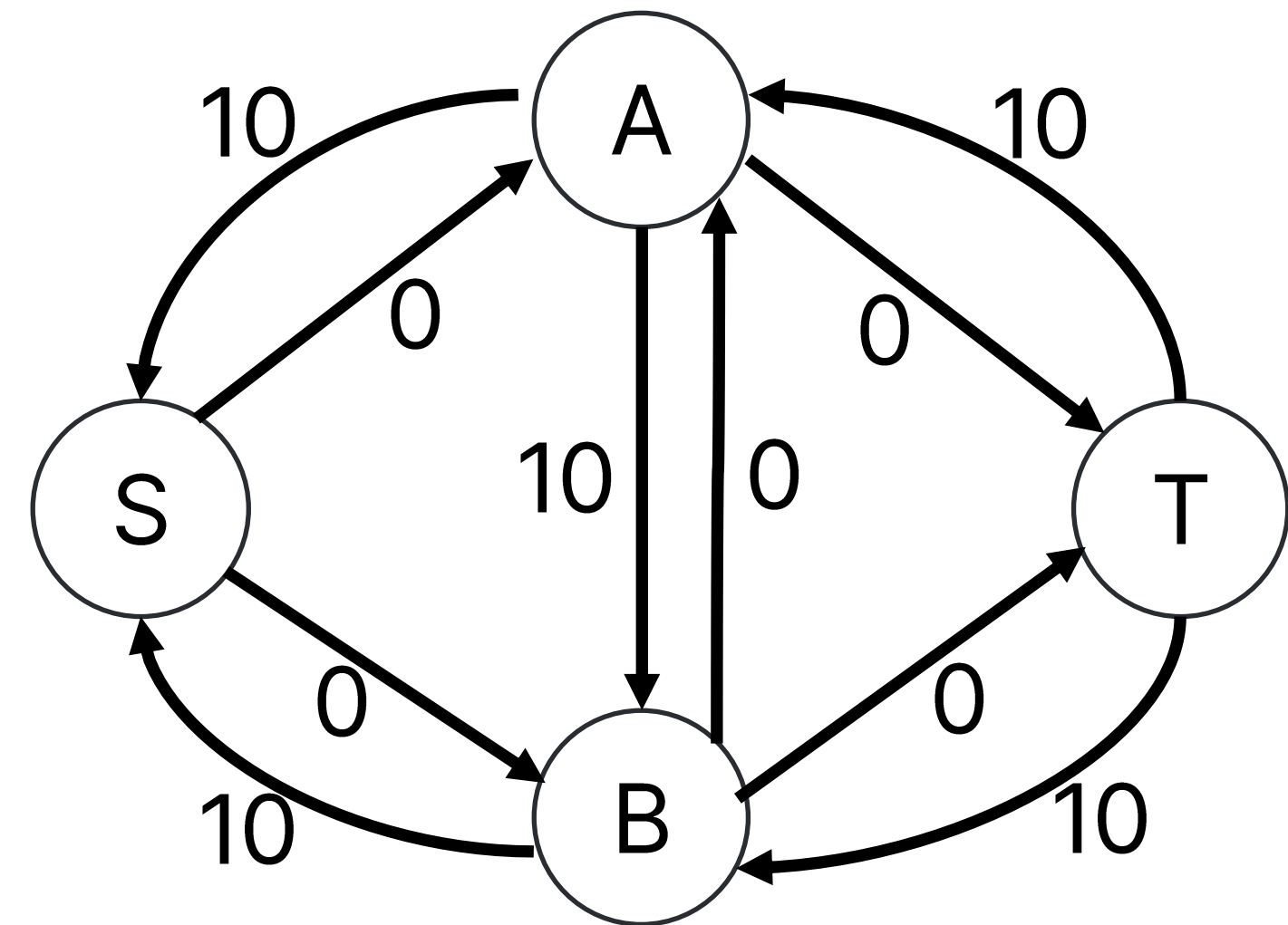
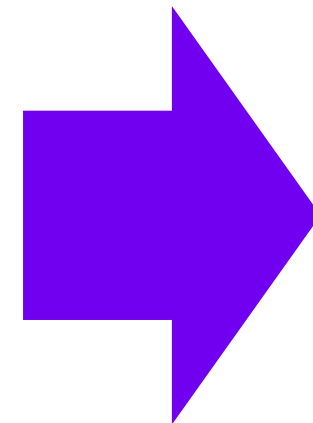
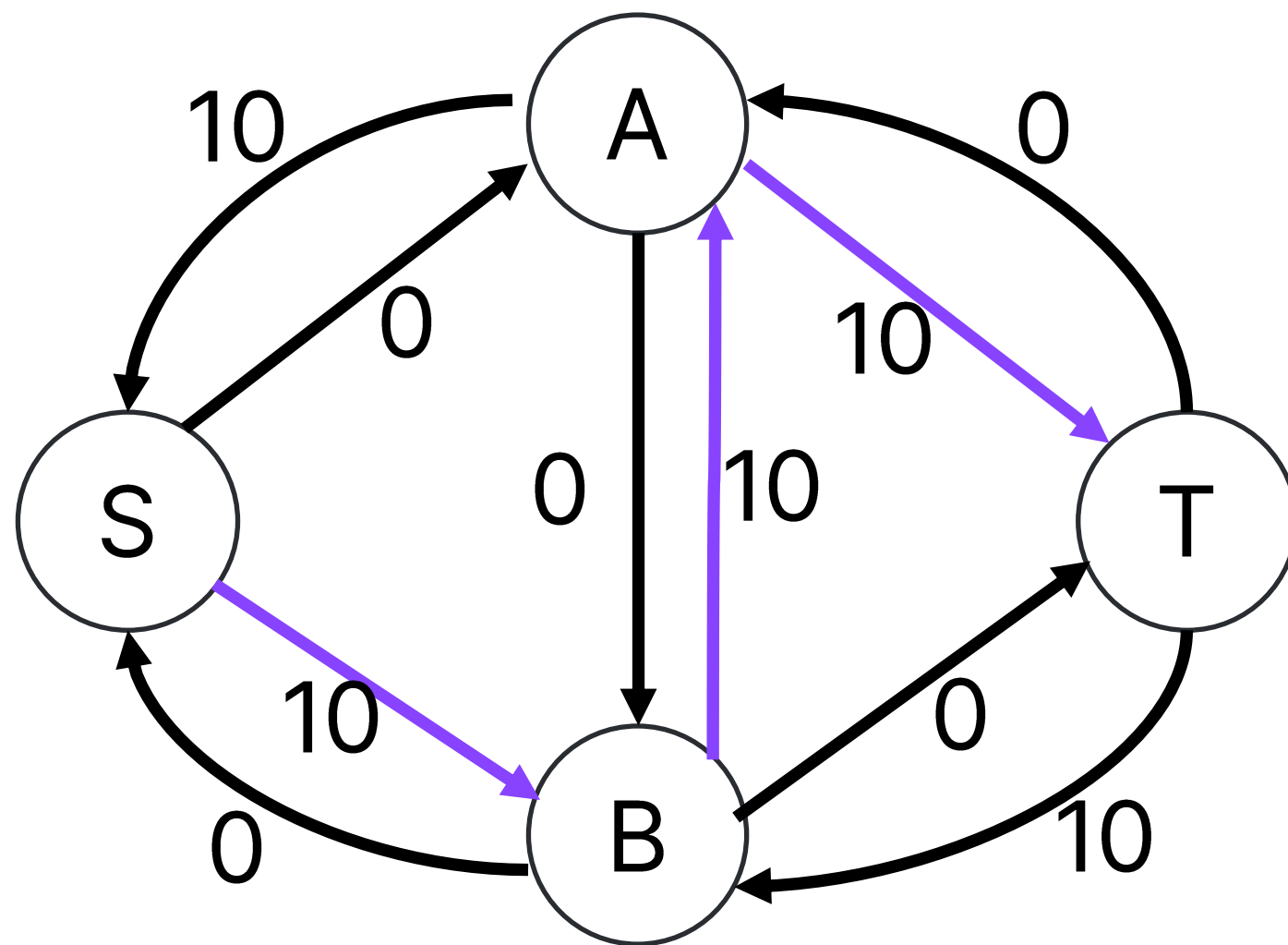
증가경로 발견:  $S \rightarrow A \rightarrow B \rightarrow T$





## Ford-Fulkerson 알고리즘 예시

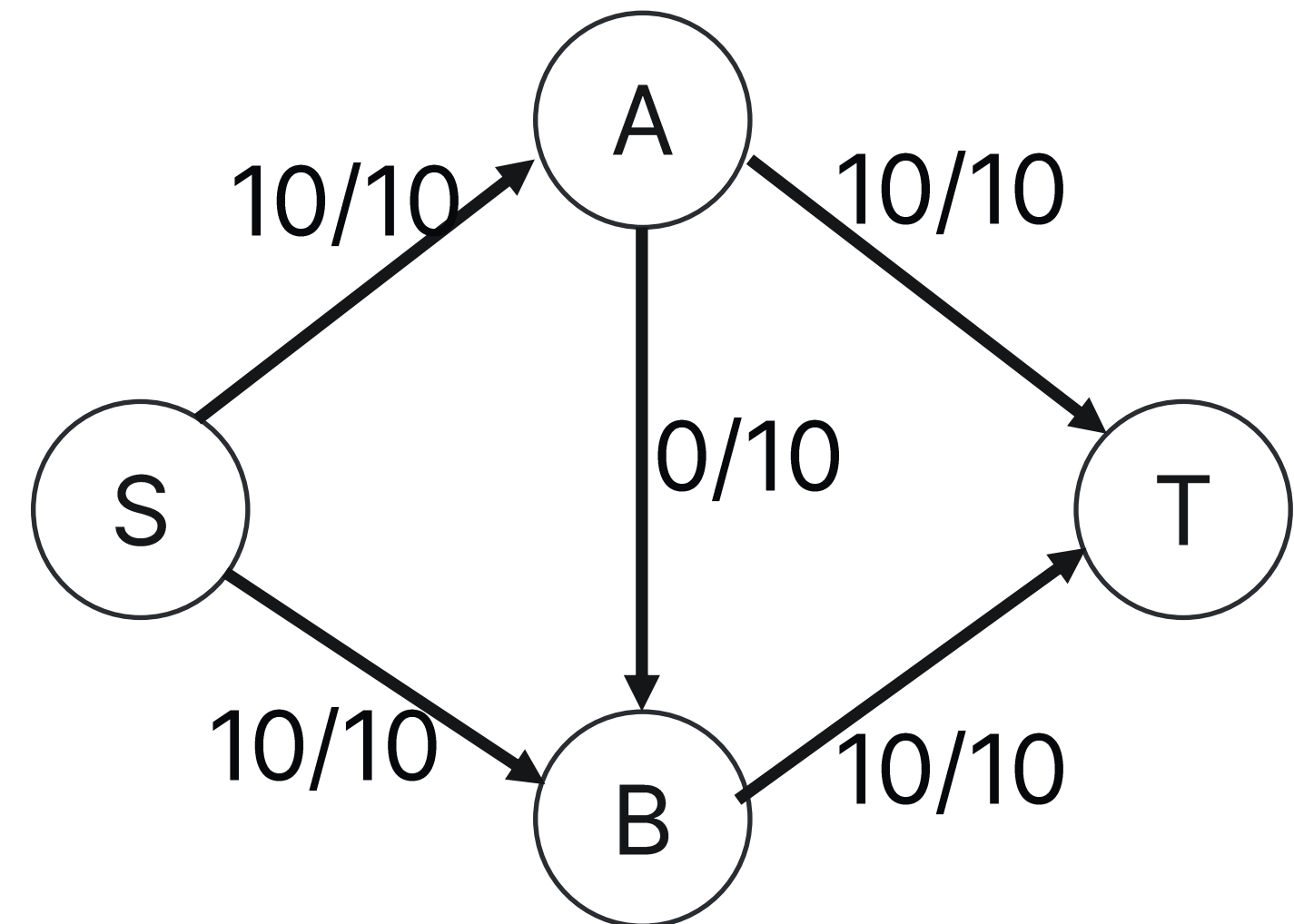
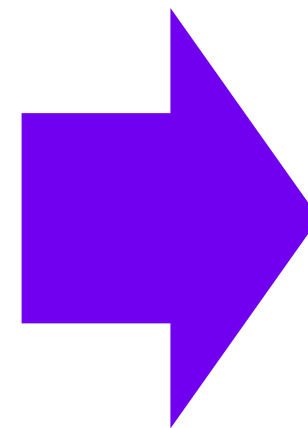
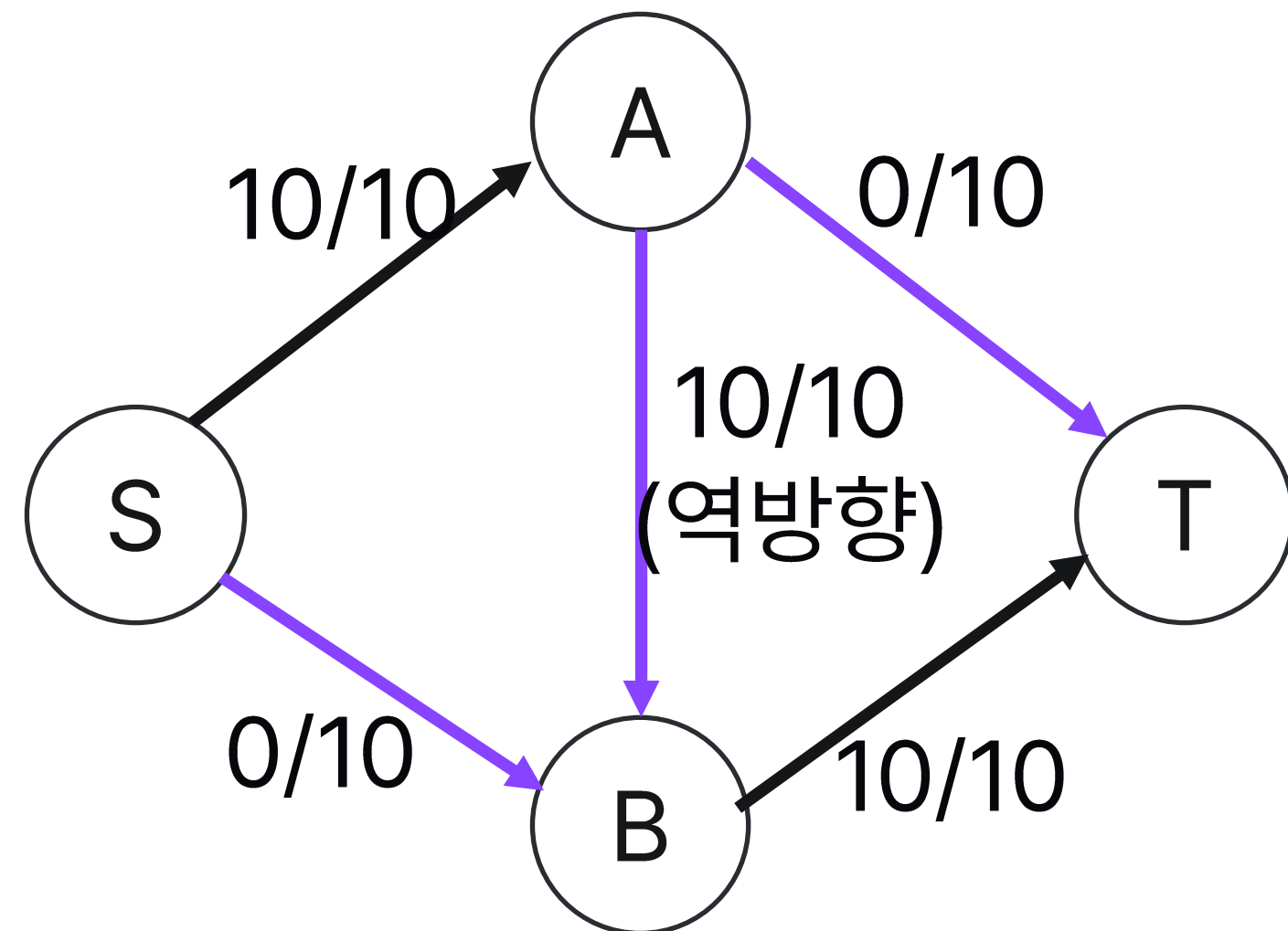
새로운 증가경로 발견:  $S \rightarrow B \rightarrow (\text{역방향 잔여 용량 } 10) \rightarrow A \rightarrow T$





## Ford-Fulkerson 알고리즘 예시

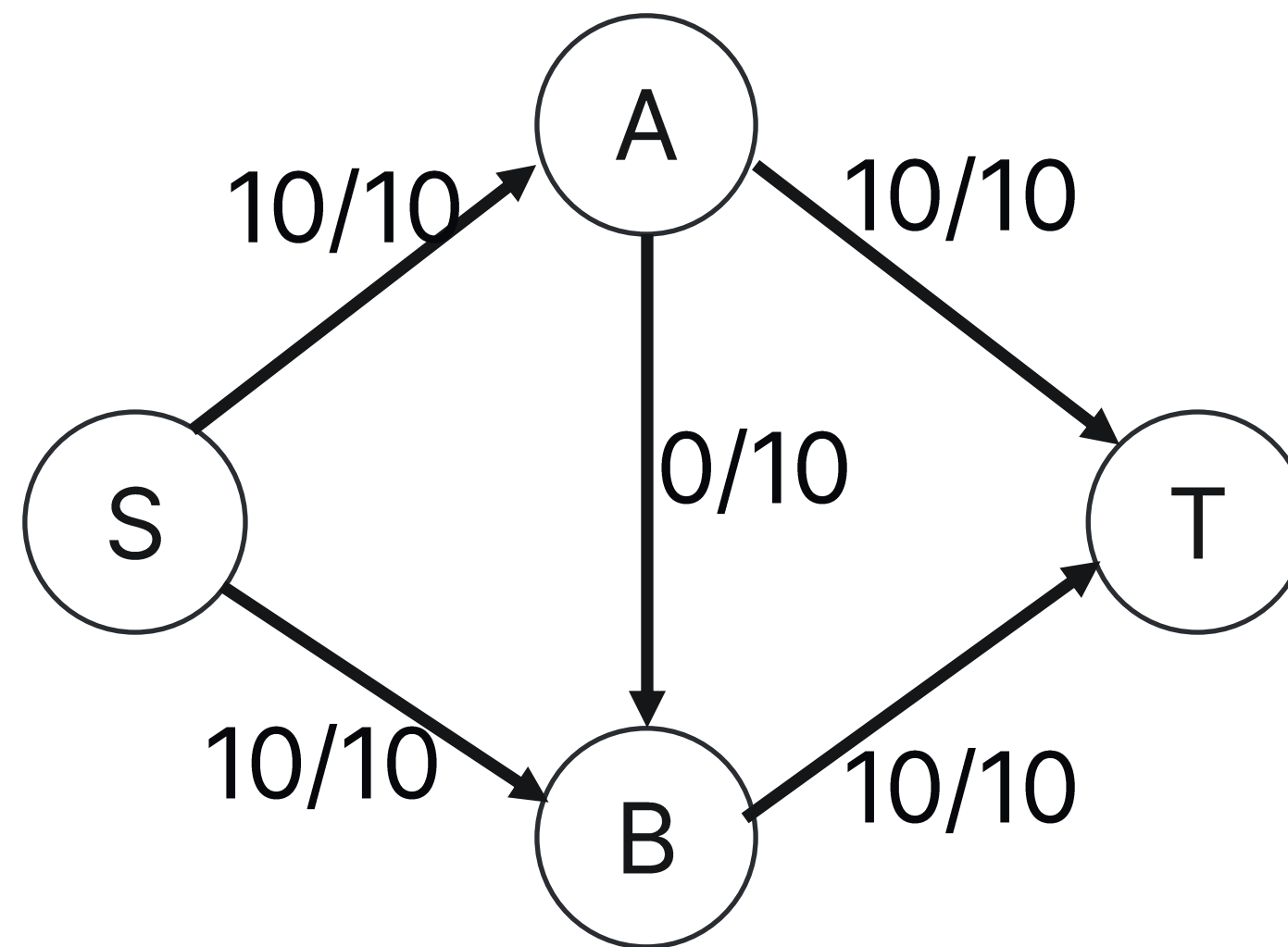
새로운 증가경로 발견:  $S \rightarrow B \rightarrow (\text{역방향 잔여 용량 } 10) \rightarrow A \rightarrow T$





## Ford-Fulkerson 알고리즘 예시

최종 유량:  $10(S \rightarrow A \rightarrow T) + 10(S \rightarrow B \rightarrow T)$





## Ford-Fulkerson 알고리즘 구현

- 잔여그래프에서 DFS를 사용하여 증가경로의 존재여부 확인
- 방문체크 배열(visited)을 사용하여, 방문하지 않은 정점 중 잔여용량이 0보다 큰 곳으로만 이동 배열을 사용하여 이동
- 시간 복잡도:  $O(f \cdot E)$ 
  - $E$ (탐색 비용): 최악의 경우 모든 간선을 한번씩 탐색
  - $f$ (최대 유량 값): 한번의 경로 탐색으로 최소 1이상의 유량 보냄 → 최악의 경우 총  $f$  번 DFS 반복
  - 시간복잡도가  $f$ (최대 유량값) 에 의존



## Ford-Fulkerson 알고리즘 구현

1. 잔여그래프를 생성합니다.

→  $\text{capacity}[u][v]$ 는 현재 상태에서  $u \rightarrow v$  로 더 보낼 수 있는 양

```
def solve_ford_fulkerson_dfs():  
    n = 4  
    source, sink = 0, 3  
    edges = [(0, 1, 1), (0, 2, 1), (1, 2, 1),  
             (1, 3, 1), (2, 3, 1)]  
    total_flow = 0  
  
    # 1. 잔여그래프(capacity)를 생성하시오  
    capacity = [[0] * n for _ in range(n)]  
    for u, v, cap in edges:  
        capacity[u][v] = cap
```



## Ford-Fulkerson 알고리즘 구현

2. 잔여 그래프에서 증가 경로를 찾는 DFS 함수를 구현합니다.

```
# 2. 현재 노드(curr), 도착지 노드(target),  
# 방문노드(visited),  
# 경로(path)에 대하여 DFS 탐색을 수행하는 dfs 함수를  
# 작성하시오  
def dfs(curr, target, visited, path):  
    visited[curr] = True  
    path.append(curr)  
  
    if curr == target:  
        return True  
  
    for next_node in range(n):  
        # 방문하지 않았고 잔여 용량이 0보다 큰 경로 탐색  
        if not visited[next_node] and  
        capacity[curr][next_node] > 0:  
            if dfs(next_node, target,  
                visited, path):  
                return True  
  
    path.pop() # 경로가 아니면 백트래킹  
    return False
```



## Ford-Fulkerson 알고리즘 구현

- 잔여 그래프에서 증가 경로를 하나 찾아 병목 유량만큼 유량을 흘립니다.
- 경로가 더 이상 없을 때까지 반복합니다.

```
# 경로가 더 이상 없을 때까지 반복
while True:
    visited = [False] * n
    path = []

    # DFS를 통해 증가 경로(path)를 찾음
    if not dfs(source, sink,
                visited, path):
        break # 더 이상 경로가 없으면 종료

# 3. 찾은 경로에서의 병목
# 용량(bottleneck)을 계산
bottleneck =
min(capacity[path[i]][path[i
+ 1]]) for i in
range(len(path) - 1))
```

```
# 4. 잔여 그래프(capacity)에서 정방향과
# 역방향을 업데이트

for i in range(len(path) - 1):
    u, v = path[i], path[i + 1]
    capacity[u][v] -= bottleneck
    capacity[v][u] += bottleneck

total_flow += bottleneck
```

# Edmonds-Karp 알고리즘



## Ford-Fulkerson 알고리즘의 특징

- Ford-Fulkerson 시간 복잡도:  $O(f \cdot E)$ 
  - $f$ (최대 유량값) 와  $E$ (간선의 개수) 에 의존
  - 최악의 경우 총  $f$ (최대 유량값) 번 DFS 반복

➔ 구현이 간단하나, 유량이 큰 경우 속도가 느려질 수 있다!



## Edmonds-Karp 알고리즘의 등장

- Ford-Fulkerson과 달리, BFS를 적용하여 "가장 단계가 적은(최단거리) 경로" 부터 탐색
- Edmonds-Karp 시간 복잡도:  $O(V \cdot E^2)$ 
  - $V$ (정점의 개수)와  $E$ (간선의 개수)에 의존

➔ 아무리 유량이 커도, 일정한 시간내에 끝나는 것을 수학적으로 보장



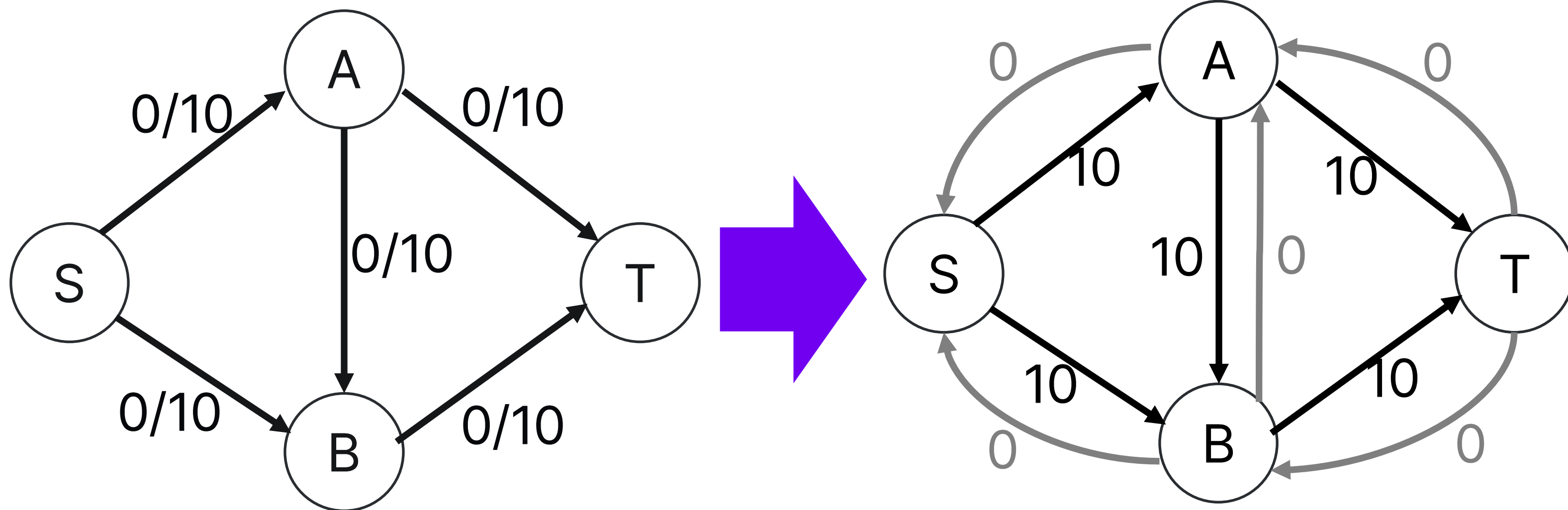
## Edmonds-Karp 알고리즘 절차

1. 처음엔 모든 유량을 0으로 설정합니다
2. 잔여 그래프에서 BFS를 이용해 최단 증가 경로를 하나 찾습니다
3. 찾은 경로의 병목 용량만큼 유량을 흘립니다
4. 경로가 더 이상 없을 때까지 반복합니다



## Edmonds-Karp 알고리즘 예시

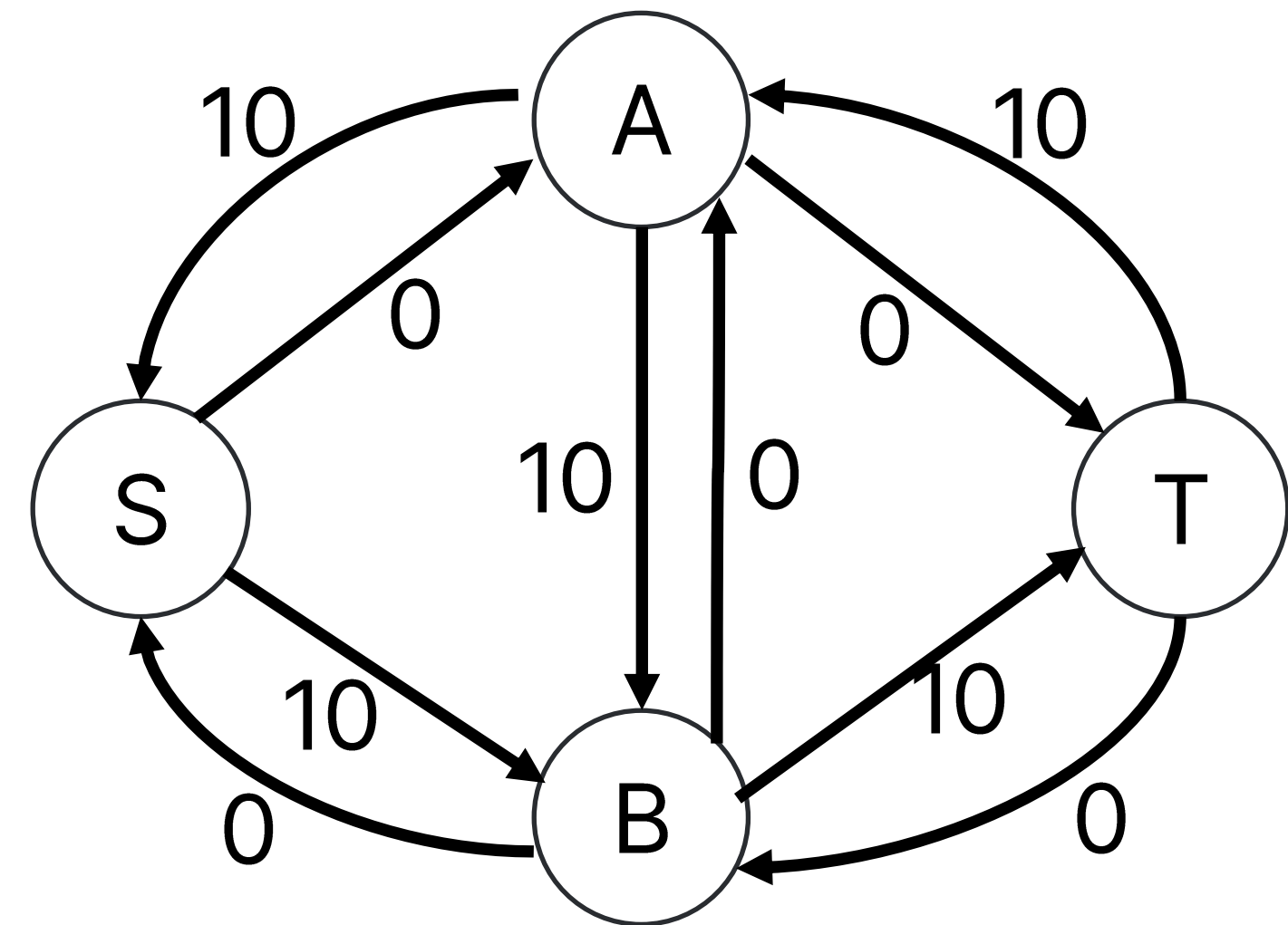
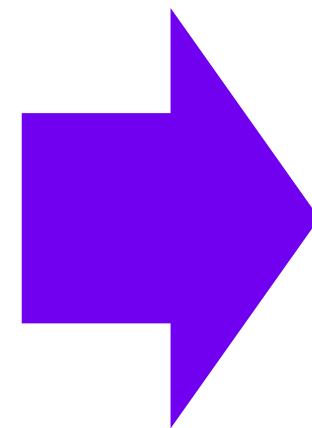
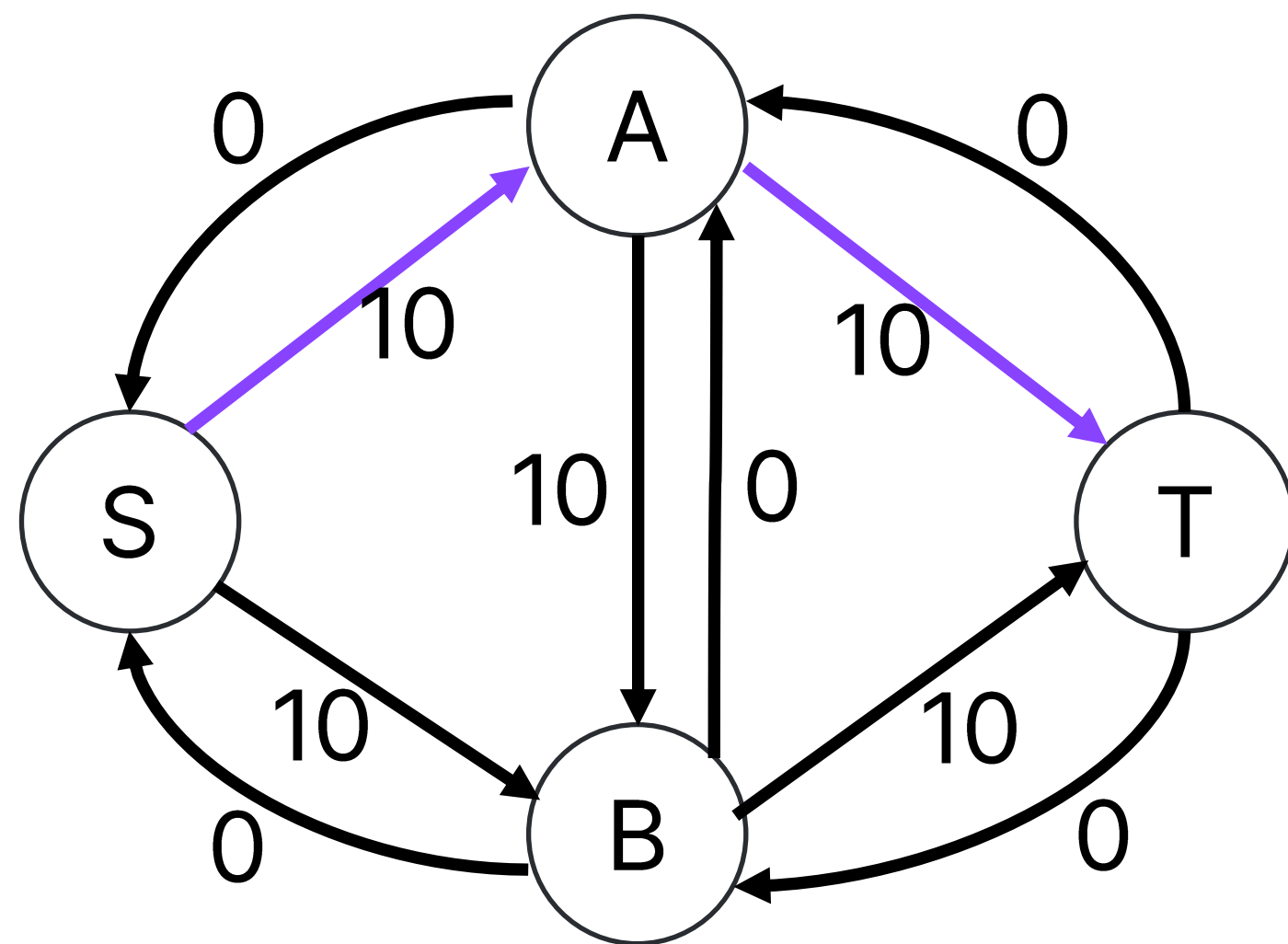
초기 상태: 잔여 그래프로 변환





## Edmonds-Karp 알고리즘 예시

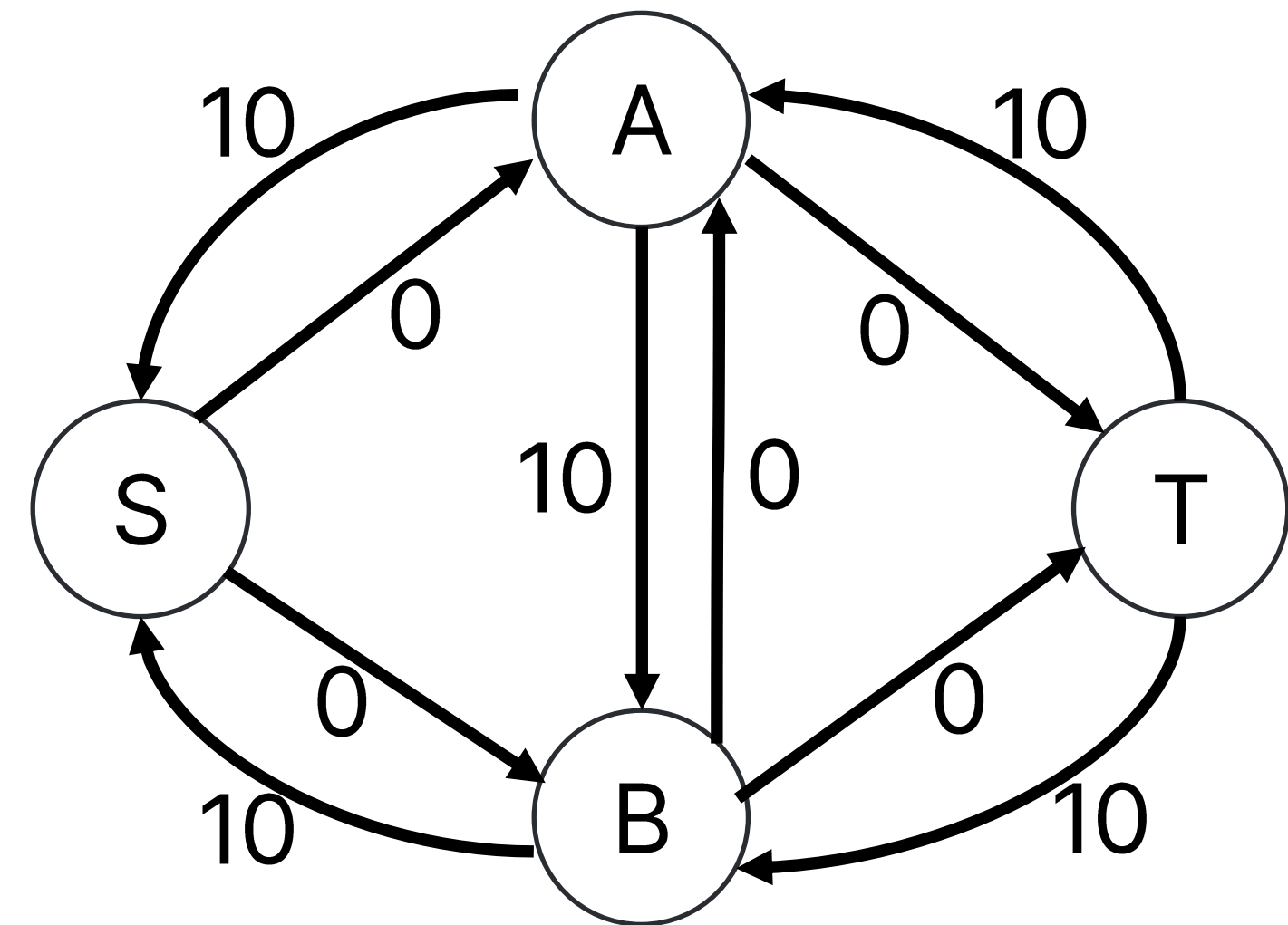
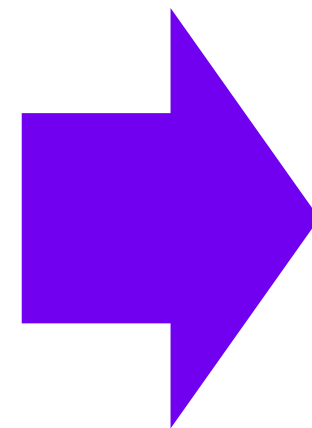
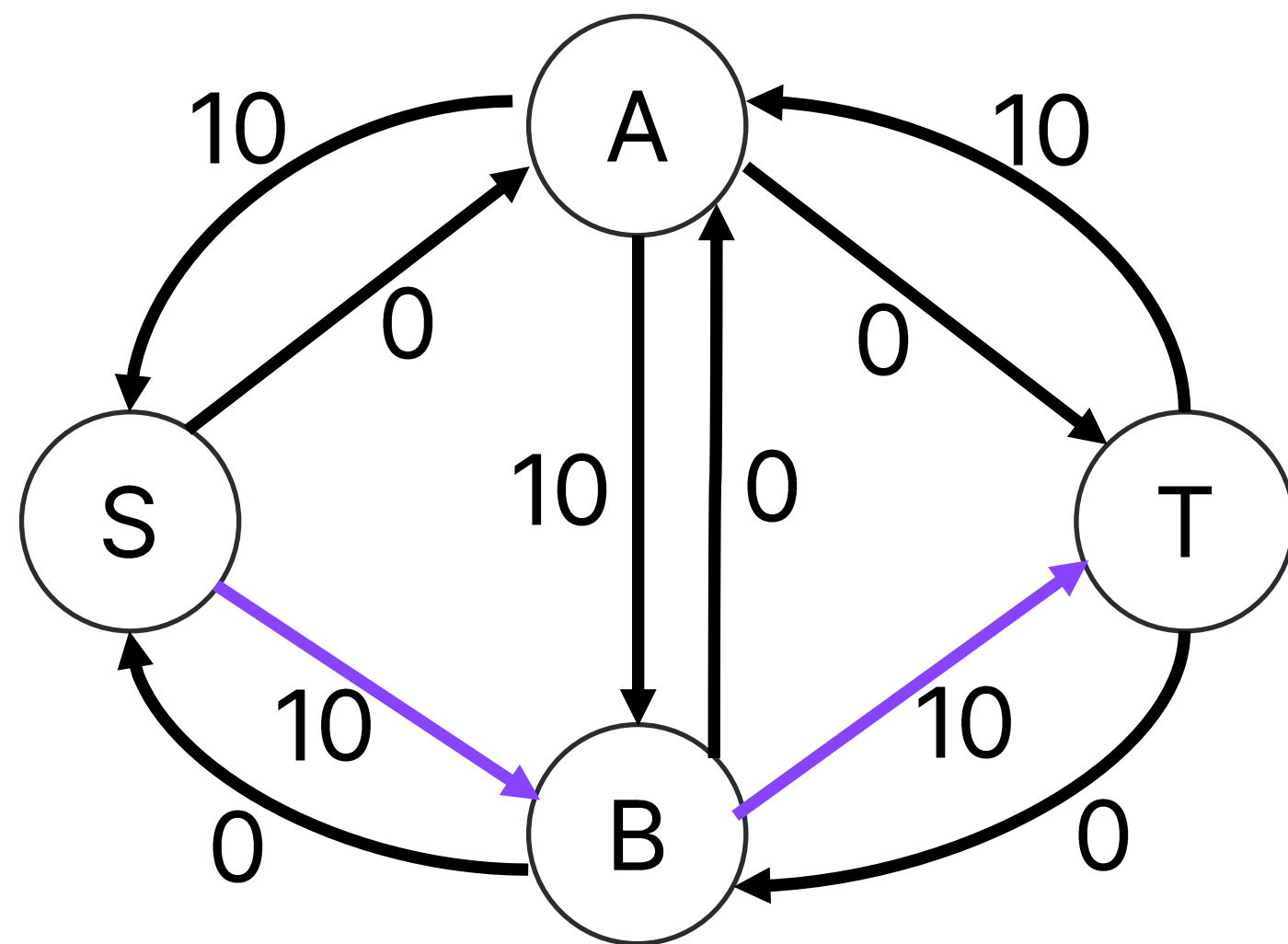
증가경로(최단경로) 발견:  $S \rightarrow A \rightarrow T$





## Edmonds-Karp 알고리즘 예시

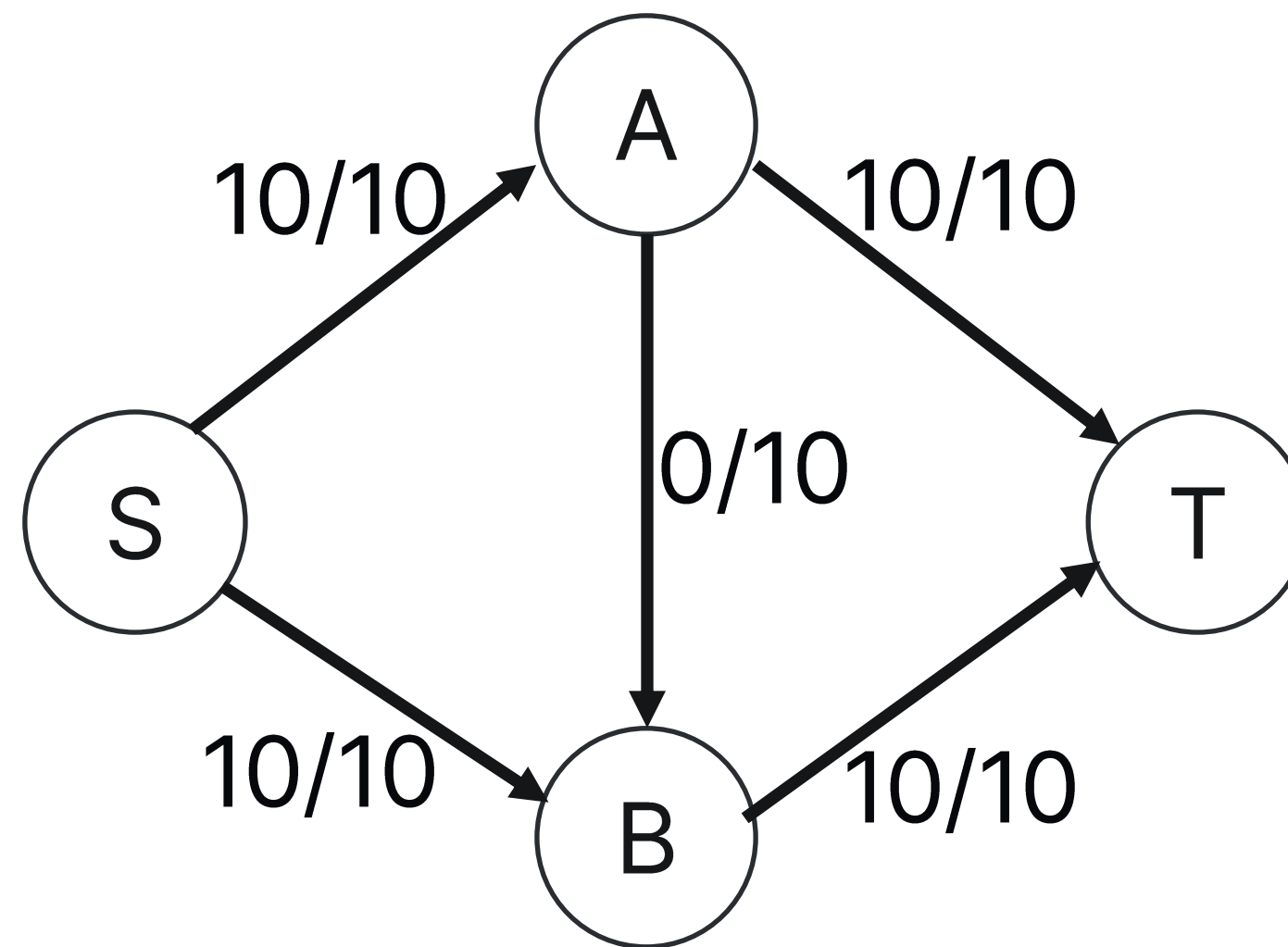
증가경로(최단경로) 발견:  $S \rightarrow B \rightarrow T$





## Edmonds-Karp 알고리즘 예시

최종 유량:  $10(S \rightarrow A \rightarrow T) + 10(S \rightarrow B \rightarrow T)$





## Edmonds-Karp 알고리즘 구현

1. 잔여그래프를 생성합니다.

→  $\text{capacity}[u][v]$ 는 현재 상태에서  $u \rightarrow v$  로 더 보낼 수 있는 양

```
def solve_edmonds_karp_bfs():  
    n = 4  
    source, sink = 0, 3  
    edges = [(0, 1, 1), (0, 2, 1), (1, 2, 1), (1, 3, 1), (2, 3, 1)]  
    total_flow = 0  
  
    # 1. 잔여그래프(capacity)를 생성하시오  
    capacity = [[0] * n for _ in range(n)]  
    for u, v, cap in edges:  
        capacity[u][v] = cap
```



## Edmonds-Karp 알고리즘 구현

### 2.1 큐를 이용하여 "간선수가 가장 적은 최단경로" 를 탐색하는 BFS함수 구현

```
def bfs(s, t, path):  
    # 각 정점 도달 전 부모 노드를 기록하기 위한 배열 (방문 체크  
    # 용)  
    parent = [-1] * n  
    queue = deque([s])  
    parent[s] = s # 시작점 방문 표시  
  
    while queue:  
        curr = queue.popleft()  
        if curr == t:  
            break # 싱크 도달 시 탐색 중단  
  
        for next_node in range(n):  
            # 방문하지 않고 잔여 용량이 남아있는 간선 탐색  
            if parent[next_node] == -1 and  
               capacity[curr][next_node] > 0:  
                parent[next_node] = curr # 탐색  
                # 과정에서 부모 노드 기록  
                queue.append(next_node)
```



## Edmonds-Karp 알고리즘 구현

2.2 큐는 경로를 저장하지 않으므로, “**parent 배열**”을 만들어 각 정점에 도달하기 직전의 부모 노드를 명시적으로 기록 → 싱크 도달 후, 역추적 하며 순차적으로 갱신

```
# 싱크( t ) 에 도달하지 못한 경우
if parent[ t ] == - 1 :
    return False

# 부모 기록을 역추적하여 경로( path ) 리스트 완성
curr = t
while curr != s :
    path.append( curr )
    curr = parent[ curr ]
path.append( s )
path.reverse() # Source ->
Sink 순서로 정렬
return True
```



## Edmonds-Karp 알고리즘 구현

- 잔여 그래프에서 증가 경로를 하나 찾아 병목 용량만큼 유량을 흘립니다.
- 경로가 더 이상 없을 때까지 반복합니다.

```
# BFS를 통해 최단 증가 경로(path)를 찾음
if not bfs(source, sink, path):
    break # 더 이상 경로가 없으면 종료

# 3. 찾은 경로에서의 병목 용량(bottleneck)을 계산
# 하시오
bottleneck =
min(capacity[path[i]][path[i+1]] for i
in range(len(path)-1))

# 4. 잔여 그래프(capacity)에서 정방향과 역방향을 업데이트
# 하시오
for i in range(len(path)-1):
    u, v = path[i], path[i+1]
    capacity[u][v] -= bottleneck # 정방향
    # 유량 감소
    capacity[v][u] += bottleneck # 역방향
    # 유량 증가 (상쇄)
```

```
total_flow += bottleneck # 최대 유량 합산
```